

基于守恒有限体积法的圆柱药材 热湿传递与烘干时间预测

摘要

针对圆柱药材的温湿分布、烘干终点及收缩影响，建立理想化、有效参数的一维径向热湿传递模型。采用守恒有限体积法与隐式 BDF 积分，以径向最大干基含水率低于 0.15 kg/kg 作为完成条件。

针对问题一，将常热物性与含水率相关扩散函数代入公共模型，在表面加密网格上计算预热过程。0.50 小时中心、表面温度为 33.5753 、 36.7856 摄氏度，含水率为 2.5500 、 1.5102 kg/kg 。特征尺度比较呈现相同的响应先后关系，水分收支检查满足内部一致性。

针对问题二，从初态统一采用温湿相关物性，联立积分全过程并摘录前三小时结果。3 小时径向温差为 0.1169 摄氏度，中心、表面含水率仍为 1.7662 、 1.0081 kg/kg 。结合局部扩散率变化与加密比较，水分均匀化滞后于升温的结果在当前数值设置下稳定。

针对问题三，在问题二的解上定位最大含水率越阈事件，得到 57.4728 小时。表面、均值与最大值的对照表明，完成判据需覆盖较慢脱水的位置；网格与时间容差复验检查了事件稳定性。

针对问题四，由干质量守恒建立均匀径向收缩下的随体模型，在固定归一化网格上求解，得到 51.0908 小时，终点半径 1.2000 厘米。总体时间比问题三缩短 11.1043% ；保持问题四物性而固定半径的对照为 129.8449 小时，同物性下收缩降幅为 60.6524% 。移动域非均匀解析基准检查了变化几何因子的离散处理。

固定域与移动域解析算例、几何因子对照、水分平衡及加密检验共同支持数值格式与结果稳定性，四问统一采用 640 个径向区间。所得时间限定于有效边界、忽略潜热、4 小时后 50 摄氏度与 0.05 kg/kg 等约定；现有资料未提供内部实验对照。

关键词：药材烘干；热湿耦合；径向传递模型；有限体积法；最大含水率阈值

1 问题重述

1.1 问题背景

热风烘干通过药材表面传入热量并排出水分，温度响应与内部脱水可能具有不同的时间尺度。题设药材近似为长 25 厘米、初始半径 2 厘米的圆柱体，初始温度 28 摄氏度、干基含水率 2.55 kg/kg。已知烘房初期温湿记录、材料经验参数及外半径变化记录，要求描述径向温湿变化并确定烘干时间。

1.2 问题要求

问题一采用原题附录 2 的参数，求解前 1800 秒的温度和含水率，按指定时刻、位置填写题表，并输出逐秒、每 0.1 厘米的完整结果。问题二统一采用原题附录 3 的经验公式描述整个烘干过程，给出前三小时每隔 0.5 小时的题表及全过程逐秒结果。

问题三沿用问题二的模型，以药材各处含水率均低于 0.15 kg/kg 为要求确定完成时间，按每 6 小时和每 60 秒分别给出题表及完整结果。问题四改用原题附录 4 的物性和附件 2 的变化半径，计算收缩过程的完成时间及含水率分布，输出时区分固定位置与当前表面。

2 问题分析

2.1 问题一分析

短时预热中，表面随环境变化而内部响应滞后，空间平均量难以回答题目指定位置的要求。采用圆柱径向导热与扩散模型，输入前 1800 秒的环境记录和常热物性，输出两场的径向分布；含水率相关扩散系数用于描述失水后的迁移减慢。

2.2 问题二分析

温度与含水率共同改变扩散率，含水率还改变热容量和导热率，需联立求解两场。将整组物性从初始时刻统一替换为原题附录 3 的函数，保留固定几何与表面交换形式。前三小时剖面用于分析耦合过程，完整轨迹供问题三确定终点。

2.3 问题三分析

表面和平均含水率较早下降，判定“各处”达标的关键是覆盖较慢脱水的位置。对问题二的全场解取径向最大值，定位其向下越过阈值的时刻；同时比较表面值和均值，说明不同指标对完成时间的影响。长期计算所需的尾段环境以明确假设补足。

2.4 问题四分析

外半径记录给出了运动边界，却未直接给出内部材料点的运动。采用长度不变的均匀径向收缩，以随体坐标将移动域映射到固定区间，并由干质量守恒推导含水率方程。问题三、四同时改变物性和几何，因此另用问题四物性下的固定半径结果作条件对照，区分总体差异与几何影响。

2.5 各问题之间的关系

四问共享初态、表面交换形式和数值框架，具体继承关系见表1。问题二、三是同一个初边值问题的不同输出；问题一作为独立的短时工况，不与问题二按时间拼接。第5章给出公共模型，第6至9章分别写各问的具体化与结果。

表 1: 四问的模型变化、求解目标与结果对应

问题	物性与几何	核心求解目标	正文数值表
一	常热物性, $D(C)$, 固定半径	前 1800 秒温度、水分径向场	5、6
二	温湿相关物性, 固定半径	全过程耦合场及前三小时摘录	7、8
三	完全沿用问题二	全域含水率首次越阈及整秒终点	9
四	另一组物性, 给定 $R(t)$	移动域越阈、实际表面及域内输出	10

3 模型假设

研究采用理想化、有效参数的一维径向传热传质模型，目标是在题给数据和约定条件下描述变化规律。表2集中列出各项假设的作用、条件及主要影响。

表 2: 公共假设、建模作用及适用条件

对象	假设与采用原因	条件及主要影响
一维径向场	轴向、周向均匀, 只保留中心至表面的差异, 以圆柱对称性简化求解	初态和侧面工况近似均匀, 轴向及端部影响较弱; “全域”限于径向场
有效边界	缺少吸附等温线, 定义 $C_e = W_a$, 使附件记录与 h_m 闭合表面驱动力	属于等数值映射; 可能改变表层失水速率和长期时间, 偏差方向取决于未知平衡关系
能量简化	忽略蒸发潜热及内部热源, 采用有效热容量描述升温	可能低估蒸发冷却的影响; 对总干燥时间的净作用取决于热湿耦合
两类密度	ρ_h 只组成 $A = \rho_h c_p$, ρ_d 用于质量守恒, 使经验容量与干基收支各有定义	两者含义和来源不同, 不附加 $\rho_h = (1+C)\rho_d$ 关系
尾段环境	记录仅到 4 小时, 此后规定 50 摄氏度和 0.05 kg/kg; 传递系数不变	用于补足长期边界, 尾段预测受该约定影响; 已有情景扰动见第10章
收缩几何	问题四均匀径向收缩、长度不变, 使给定 $R(t)$ 可确定内部运动	固定 $x = r/R(t)$ 对应材料点; 局部不均匀收缩和开裂超出此近似
干固体	初始干密度空间均匀, 干质量不流失, 以建立干基水分收支	均匀收缩下 $\rho_d R^2$ 为常数; 绝对质量仍需初始质量尺度

4 符号说明

原题水分浓度即干基含水率 C , 表示水质量与干物质质量之比, 湿基质量分数则为 $C/(1+C)$ 。表3统一全文主要符号, 含水率阈值及结果表均沿用干基定义。

$\rho_h(C)$ 的数值函数来自原题, ρ_d 是为明确质量守恒引入的变量。前者只用于热方程, 后者表示当前单位体积中的干物质质量; 均匀干密度下, 正参考值 ρ_{d0} 可从含水率方程约去, 无须另赋数值。 W_a 的单位沿用附件标注, C_e 按材料干基定义; 二者数值相等是表2规定的边界闭合, 而非材料吸附平衡的识别结果。

表 3: 主要符号、含义与单位

符号	含义	单位
θ, θ_∞	材料温度、环境温度	摄氏度
$T_K = \theta + 273.15$	材料绝对温度	K
C, C_s	材料干基含水率及其表面值	kg 水/kg 干物质
W_a	题给环境水分记录	kg/kg (附件标注)
C_e	映射到材料干基的有效边界值	kg 水/kg 干物质
$\bar{C}, C_{\max}$	径向体积权均值、最大含水率	kg 水/kg 干物质
t, t_*, t_{int}	时间、越阈事件、严格整秒终点	s
r, R, R_0	径向位置、当前及初始外半径	m
$x = r/R$	归一化半径	1
L, v	圆柱长度、材料径向速度	m、m/s
ρ_h	题给热容量经验参数	kg/m ³
ρ_d, ρ_{d0}	当前干物质密度、初始参考值	kg 干物质/m ³
c_p	题给比热	J/(kg K)
$A = \rho_h c_p$	有效体积热容量	J/(m ³ K)
k, D	导热率、水分扩散系数	W/(m K)、m ² /s
h, h_m	对流换热、有效传质系数	W/(m ² K)、m/s
M_d, M_w	干物质质量、水质量	kg
N, w_i	径向区间数、控制体径向权重	1
Λ	单位干质量的累计水分损失	kg 水/kg 干物质

5 数据处理与模型准备

5.1 数据来源与预处理

附件 1 含 241 条环境记录，覆盖 0 至 4 小时，间隔 60 秒；附件 2 含 145 条半径记录，覆盖 0 至 72 小时，间隔 1800 秒。时间严格递增，无缺失、非有限值或重复时刻。半径非增，含 53 个不同取值和 92 次相邻重复，平台记录全部保留。计算统一用秒和米，展示时换算为小时或厘米；物性系数按原题单位使用，输入输出见表4。

表 4: 原始附件与输出要求

对象	记录数	时间范围/h	角色与处理
附件 1	241	0–4	温度与有效边界含水率；全量线性插值
附件 2	145	0–72	题给半径；厘米转米，保留全部平台记录
结果模板 1、2	两工作表	依各问要求	温度、水分分别输出；每秒和每 0.1 厘米
结果模板 3、4	单工作表	至严格终点	含水率每 60 秒输出，并补入最终整秒

图1展示环境记录。相邻时刻间按式(1)线性插值, b_j 为第 j 条记录值, 保留各端点及局部变化。

$$b(t) = b_j + \frac{t - t_j}{t_{j+1} - t_j}(b_{j+1} - b_j), \quad t_j \leq t \leq t_{j+1}, \quad b \in \{\theta_\infty, C_e\}. \quad (1)$$

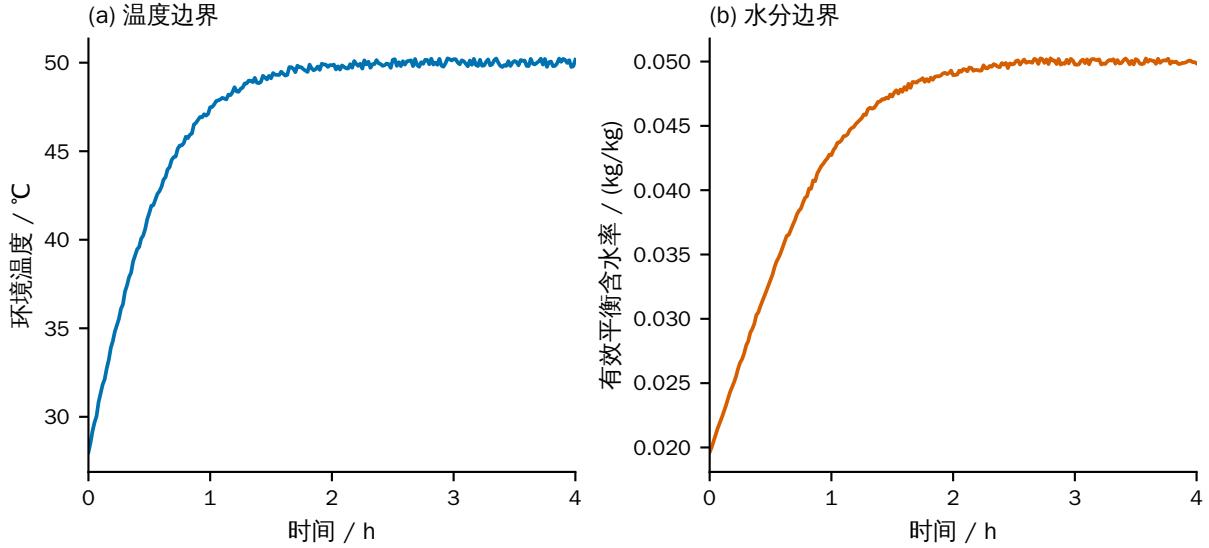


图 1: 环境温度与有效边界含水率: 全部 241 条记录及分段线性插值, 两种变量分别展示。

5.2 公共理论与作用机理

初始区域为 $0 \leq r \leq R_0 = 0.02$ 米, 长度 $L = 0.25$ 米。热量经侧面传入, 在径向温度梯度下向内部传播; 水分则在含水率梯度驱动下向表面迁移。柱坐标下薄环控制体的体积随半径变化, 方程需保留相应的径向权重。

依据题给热、质传递系数, 表面采用保留有限外部阻力的 Robin 条件。圆柱干燥模型中对不同边界形式的比较见文献 [1], 此处只借鉴边界形式, 系数和物性仍以原题为准。共享面通量的有限体积方法适合描述柱状介质热湿耦合 [2], 也便于核对控制体收支。

热湿反馈由经验函数体现: 温度通过 $D(\theta, C)$ 影响迁移, 水分通过 D 、 k 和 A 影响扩散与升温。常热物性且 D 只含 C 时, 两场可分别求解; 物性同时依赖温湿状态时, 采用联立求解。各问的具体函数分别列于对应章节。

5.3 基础控制方程

以导热通量 $-k(C)\theta_r$ 表示相邻薄环之间的热传递, 净流入热量引起温度变化。固定域的热湿控制方程合写为式(2); 第一式左端为有效容量与升温速率之积, 右端为导热通

量散度，两端单位均为 W/m^3 。

$$\begin{aligned} A(C) \frac{\partial \theta}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial \theta}{\partial r} \right), \\ \frac{\partial C}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left(r D(\theta, C) \frac{\partial C}{\partial r} \right). \end{aligned} \quad (2)$$

导热率随含水率变化，须保留在散度内部。热容量形式采用 $A(C)\theta_t$ ，若对 $A(C)\theta$ 整体求导，会增加 $\theta A'(C)C_t$ ，对应另一种能量闭合。当前形式与有效容量、忽略潜热约定一致。

式(2)第二式以干基含水率描述水分迁移。固定域干物质密度均匀且不变时，相对于固体的水分通量为 $-\rho_d D \nabla C$ ，水质量平衡除以 ρ_d 即得含水率方程。 $D(\theta, C)$ 在散度内按局部状态变化，左右两端均以含水率每秒计。

水分外排由表面通量体现，温度经题给扩散函数影响迁移。收缩域还需处理材料运动和干密度变化，第9章从相同的质量平衡推导其随体形式。

5.4 初始条件与边界条件

温湿场从式(3)的均匀初态出发。初始温度等于环境温度，热通量起初为零；含水率高于有效边界值，水分由表面排出。

$$\theta(r, 0) = 28, \quad C(r, 0) = 2.55, \quad 0 \leq r \leq R_0. \quad (3)$$

中心采用对称条件，表面保留有限热、质交换阻力，得到式(4)。下标 s 为表面值，传递系数沿用于四问，单位见表3。

$$\begin{aligned} \theta_r(0, t) &= 0, \quad C_r(0, t) = 0, \\ -k_s \theta_r(R_0, t) &= h(\theta_s - \theta_\infty), \quad h = 25, \\ -D_s C_r(R_0, t) &= h_m(C_s - C_e), \quad h_m = 8 \times 10^{-7}. \end{aligned} \quad (4)$$

取向外通量为正，升温时 $\theta_s < \theta_\infty$ ，热通量为负；脱水时 $C_s > C_e$ ，水分通量为正。均匀含水率初值与 Robin 边界在初始角点不相容，解在 $t > 0$ 形成短时表层梯度。计算保留题给初态，以早期网格和时间精度比较检查其分辨情况。

长期求解在 $t > 4$ 小时取 $\theta_\infty = 50$ 摄氏度、 $C_e = 0.05 \text{ kg/kg}$ ；4 小时整仍取末条记录 50.165 摄氏度和 0.04986 kg/kg ，随后切换至约定常值。四问全过程均取 $h = 25 \text{ W}/(\text{m}^2 \text{ K})$ 、 $h_m = 8 \times 10^{-7} \text{ m/s}$ ，4 小时后传递系数也保持不变。

5.5 评价指标与判定条件

问题一以 0.50 小时为求解终点；问题二的全过程与问题三共用终点。以下含水率指标用于描述场分布，其中最大值阈值用于问题三、四的完成判定。

“药材内部各处均低于 0.15” 对应最大值条件。式(5)分别定义径向最大含水率、体积加权均值和表面含水率。计算在全部节点上取最大值，保形径向重建保持节点间的单调性。

$$C_{\max}(t) = \max_{0 \leq r \leq R(t)} C(r, t), \quad \bar{C}(t) = \frac{2}{R(t)^2} \int_0^{R(t)} r C(r, t) dr, \quad C_s(t) = C(R(t), t). \quad (5)$$

先定位 $C_{\max} - 0.15$ 由正到负的事件，再按式(6)确定严格小于阈值的整秒输出时刻。由于题目采用严格不等式，事件恰在整数秒发生时仍取下一秒，并用未舍入的全部节点状态检查该时刻。

$$C_{\max}(t_*) = 0.15 \text{ 且向下穿越}, \quad t_{\text{int}} = \lfloor t_* \rfloor + 1, \quad C_{\max}(t_{\text{int}}) < 0.15. \quad (6)$$

5.6 数值离散和公共求解方法

计算采用归一化半径 $x = r/R$ ，固定域取 $R = R_0$ 。为分辨较陡的表层梯度，在中心至表面布置 $N + 1$ 个节点，按式(7)加密。 N 为区间数， $x_{i+1/2}$ 为控制体面， w_i 来自柱坐标积分 $\int x dx$ 。

$$x_i = 1 - \left(1 - \frac{i}{N}\right)^2, \quad i = 0, \dots, N; \quad x_{i+1/2} = \frac{x_i + x_{i+1}}{2}, \quad w_i = \frac{x_{i+1/2}^2 - x_{i-1/2}^2}{2}. \quad (7)$$

两端面取 $x_{-1/2} = 0$ 、 $x_{N+1/2} = 1$ ，权重之和为 $1/2$ 。相邻节点间的两半段阻力串联，面系数采用调和平均。对一般变量 u 及扩散系数 a ，式(8)给出含径向面权重的通量 G ， κ_b 表示相应表面传递系数。

$$\begin{aligned} a_{i+1/2} &= \frac{2a_i a_{i+1}}{a_i + a_{i+1}}, \\ G_{i+1/2} &= x_{i+1/2} a_{i+1/2} \frac{u_{i+1} - u_i}{x_{i+1} - x_i}, \\ G_{-1/2} &= 0, \quad G_{N+1/2} = R\kappa_b(u_\infty - u_N). \end{aligned} \quad (8)$$

温度取 $(u, a, \kappa_b, u_\infty) = (\theta, k, h, \theta_\infty)$ ，水分取 (C, D, h_m, C_e) 。共享面通量之差给出净交换，得到式(9)；温度方程取 $q_i = A_i$ ，水分方程取 $q_i = 1$ 。

$$\frac{du_i}{dt} = \frac{G_{i+1/2} - G_{i-1/2}}{R^2 w_i q_i}. \quad (9)$$

内部通量在径向求和时相消。中心通过控制体积分处理 $1/r$ ，由 $x_{1/2} = x_1/2$ 得到式(10)，系数 4 体现圆柱中心的几何权重。

$$\frac{du_0}{dt} = \frac{4a_{1/2}(u_1 - u_0)}{R^2 x_1^2 q_0}. \quad (10)$$

采用隐式 BDF 处理加密网格和变扩散率造成的刚性，联立积分时提供解析稀疏雅可比。状态导数同时计入通量系数和容量分母的变化，问题二的导数示例见式(14)。

最终取 640 个区间，相对容差 10^{-8} ，温度绝对容差 10^{-8} 摄氏度，含水率与累计水分损失绝对容差 10^{-10} 。前 4 小时最大步长 15 秒，此后 600 秒，实际步长自适应选择，4 小时处按边界切换分段积分。连续时间输出结合保形分段三次径向插值，得到指定位置结果。事件使用全部节点的未舍入值；保形插值保持节点间单调性，不产生超出端点值的新极值。

计算步长、结果采样间隔与数值误差分别控制。各问从未舍入解采样，题表和工作簿显示四位小数；完整输出范围、整秒终点与移动域空值规则见附录C。

6 问题一的模型建立与求解

6.1 建模思路

基于第5章的统一模型，分别求解预热阶段的温度和含水率。保持热物性不变，仅在水分方程中更新 $D(C)$ ，以中心与表面响应的差异描述短时传递过程。

6.2 变量与参数设置

半径固定为 2 厘米，初态和边界采用式(3)、(4)。材料参数取原题附录 2，见式(11)，单位沿用表3。外部工况使用前 1800 秒记录，不涉及尾段延拓。

$$\rho_h = 820, \quad c_p = 2600, \quad k = 0.36, \quad D = 7 \times 10^{-9} \exp(-0.89/C). \quad (11)$$

6.3 模型建立

将式(11)代入公共方程(2)，温度方程成为常系数径向导热方程，水分方程保留含水率相关扩散系数。两场通过同一径向几何和外部环境分别演化。本问按给定时长结束，求解目标为指定时空点的 θ 和 C 。

6.4 求解方法

采用第5章的 640 区间表面加密网格与 BDF 积分，在 0 至 1800 秒内求解。相对容差为 10^{-8} ，最大步长 15 秒；绝对容差与径向重建方法沿用公共设置。连续解按每秒、每 0.1 厘米采样，另外摘录原题指定位置形成两张题表。

6.5 结果展示与分析

表5和表6列出 100、300、600、900、1200、1500、1800 秒时, 半径 0、0.5、1.0、1.5、2.0 厘米处的温度与干基含水率。结果由最终 640 区间网格的未舍入数值解采样得到, 完整逐秒数据保存在工作簿 1 中。

表 5: 问题一指定时刻的温度 (对应题目表 1, 单位: 摄氏度)

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	28.0001	28.0003	28.0040	28.0327	28.1801
300	28.0408	28.0635	28.1514	28.3681	28.8488
600	28.4533	28.5360	28.8039	29.3159	30.1652
900	29.3243	29.4583	29.8755	30.6161	31.7304
1200	30.5427	30.7098	31.2223	32.1126	33.4276
1500	31.9957	32.1867	32.7660	33.7463	35.1203
1800	33.5753	33.7720	34.3642	35.3621	36.7856

表 6: 问题一指定时刻的干基含水率 (对应题目表 2, 单位: kg/kg)

时间/s	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

0.50 小时, 中心和表面温度分别为 33.5753、36.7856 摄氏度, 相差 3.2102 摄氏度; 中心含水率为 2.5500, 接近初值, 表面已降至 1.5102。热量已影响整个截面, 明显失水主要发生在近表面区域。

图2给出 100、600 和 1800 秒的径向剖面。温度在中心附近平坦, 向表面升高; 含水率则在表面最低, 向中心逐渐接近初值。两组剖面反映了热量传入与水分排出的不同进程。

6.6 模型检验

问题一的热扩散率为 $k/(\rho_h c_p) = 1.6886 \times 10^{-7}$ 平方米每秒, 初始水分扩散系数为 4.9377×10^{-9} 平方米每秒, 前者约为后者的 34 倍。在相同几何尺度下, 热量比水分更快影响内部。以半径定义式(12)中的 Biot 数, 热 Biot 数为 1.3889, 需保留内部温度梯度。

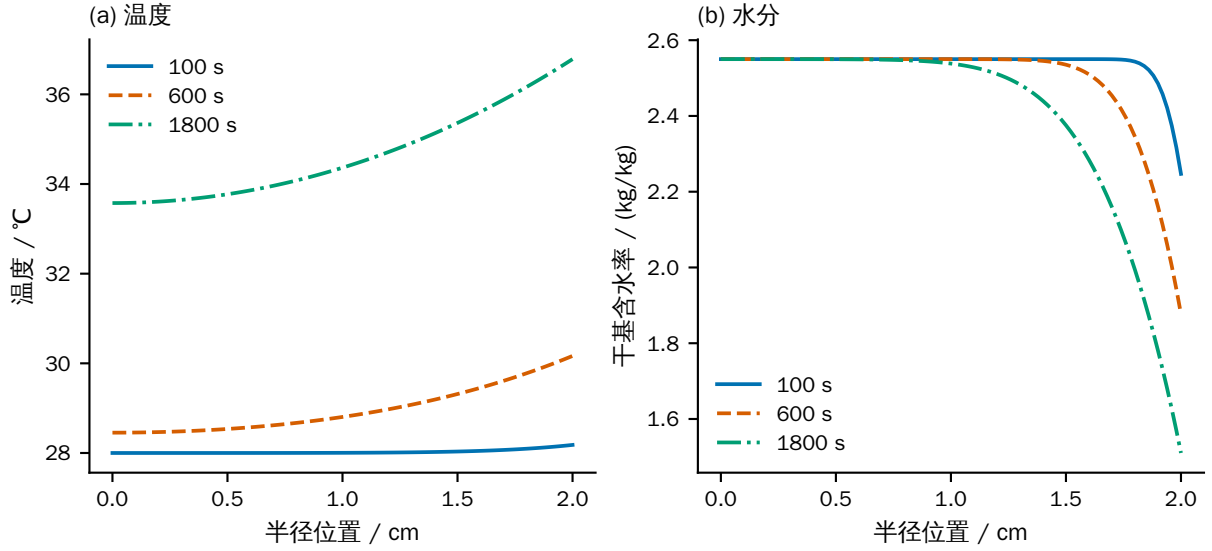


图 2: 问题一在 100、600 和 1800 秒的径向温湿剖面, 同一时刻采用相同颜色与线型。

$$\alpha = \frac{k}{\rho_h c_p}, \quad \text{Bi}_T = \frac{hR}{k}, \quad \text{Bi}_C = \frac{h_m R}{D}. \quad (12)$$

初始水分 Biot 数约为 3.2404。将 $C = 0.15$ 代入题设函数, 得到 $D = 1.8547 \times 10^{-11}$ 平方米每秒, 相应水分 Biot 数约为 862.65。这里用函数值说明低含水率时内部阻力的增大, 该函数点用于数量级对比, 0.50 小时状态仍由题表给出。计算中仍按各节点状态更新 D 。

表面含水率在短时间内下降较快, 表面状态与环境之间仍保留有限差值。Robin 条件通过题给传递系数确定交换速率, 描述表面温度和含水率对环境变化的响应。

6.7 本问结论

在题给常热物性和前 0.50 小时环境下, 中心、表面温度分别为 33.5753、36.7856 摄氏度, 含水率分别为 2.5500、1.5102 kg/kg。预热已使中心升温, 明显失水仍集中在外层, 后续求解应保留径向差异。

7 问题二的模型建立与求解

7.1 建模思路

本问将问题一的常热物性替换为温湿相关函数, 从初态联立计算全过程。中心—表面差异用于描述径向非均匀性, 局部扩散系数用于解释升温与失水的共同作用。

7.2 变量与参数设置

几何、初值和传递系数沿用公共设置，从 $t = 0$ 起统一采用原题附录 3 的式(13)。环境在前 4 小时按记录插值，此后按第5章的常值条件延拓。前三小时为题表摘录范围，完整积分与问题三的终点判定衔接。

$$\begin{aligned} \rho_h &= 650 + 128C, & c_p &= 1450 + \frac{2736C}{1+C}, \\ k &= 0.21 + \frac{0.38C}{1+C}, & D &= 0.0024 \exp\left(-\frac{0.45}{C} - \frac{3850}{T_K}\right). \end{aligned} \quad (13)$$

7.3 模型建立

将式(13)代入式(2)，在每次状态更新时同时计算 $A(C)$ 、 $k(C)$ 和 $D(\theta, C)$ ，构成双向热湿耦合。温度使用摄氏度求解，扩散指数采用绝对温度 T_K 。

图3展示问题二、三的扩散函数。温度项体现升温对扩散的促进，低含水率区的指数衰减则形成后期阻力。温度相关扩散函数也见于柱状食品热湿耦合研究 [3]，此处结构与参数仍以题目为准。

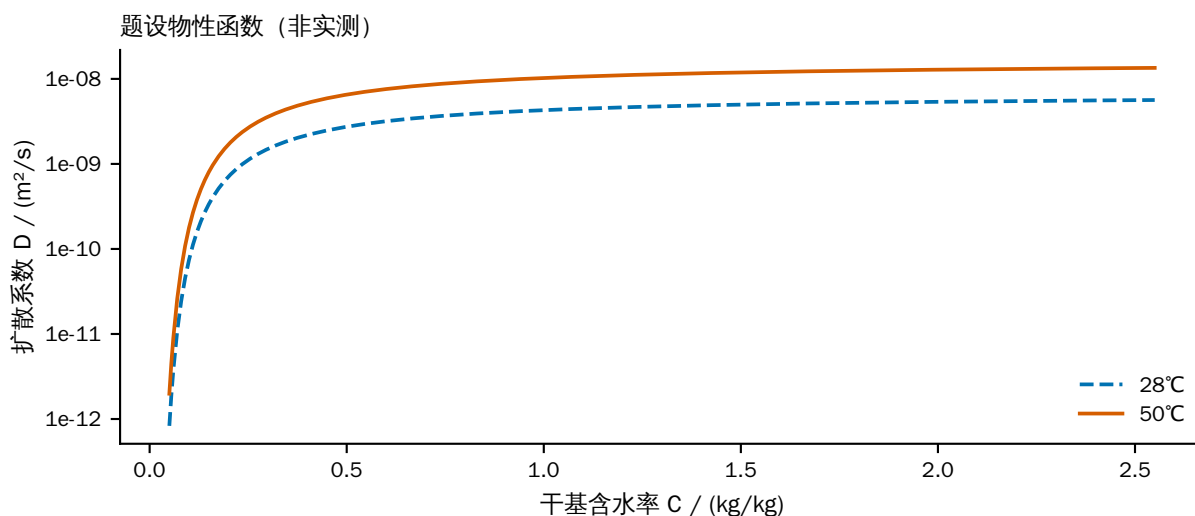


图 3: 问题二、三在 28 与 50 摄氏度下的题设扩散函数，纵轴采用对数刻度。

7.4 求解方法

沿用 640 区间网格、BDF 及第5章的容差和分段积分设置。联立系统提供解析稀疏雅可比：相邻系数 a, b 的调和平均记为 $H(a, b)$ ，其导数及本问扩散函数的导数见式(14)。温度对水分的导数还包含容量分母产生的 $-\dot{\theta}_i A'_i / A_i$ 。

$$\begin{aligned}\frac{\partial H}{\partial a} &= \frac{2b^2}{(a+b)^2}, & \frac{\partial H}{\partial b} &= \frac{2a^2}{(a+b)^2}, \\ \frac{\partial D}{\partial \theta} &= D \frac{3850}{T_K^2}, & \frac{\partial D}{\partial C} &= D \frac{0.45}{C^2}.\end{aligned}\tag{14}$$

积分至问题三确定的终点，两个工作表均输出全过程逐秒结果，表7、8只列 0.5 至 3.0 小时。输出间隔用于采样，与自适应积分步长区分。

7.5 结果展示与分析

问题二的六个指定时刻结果见表7和表8。0.50 小时，中心、表面温度分别为 32.1892、35.4130 摄氏度，均低于问题一；表面含水率为 1.6486，高于问题一的 1.5102。这一差异来自热容量、导热率和扩散函数的共同改变，传热及传质系数保持不变。

表 7: 问题二指定时刻的温度（对应题目表 3，单位：摄氏度）

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	32.1892	32.3821	32.9659	33.9608	35.4130
1	40.3816	40.5536	41.0605	41.8782	42.9976
1.5000	45.8468	45.9348	46.1930	46.6051	47.1401
2	48.4502	48.4881	48.5984	48.7736	49.0033
2.5000	49.4670	49.4792	49.5136	49.5653	49.6609
3	49.8495	49.8553	49.8746	49.9101	49.9664

表 8: 问题二指定时刻的干基含水率（对应题目表 4，单位：kg/kg）

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
0.5000	2.5499	2.5489	2.5256	2.3257	1.6486
1	2.5257	2.4948	2.3578	2.0230	1.4711
1.5000	2.3861	2.3256	2.1344	1.8020	1.3476
2	2.1709	2.1084	1.9236	1.6259	1.2311
2.5000	1.9566	1.9006	1.7360	1.4720	1.1166
3	1.7662	1.7165	1.5702	1.3333	1.0081

3 小时，中心、表面温度分别为 49.8495、49.9664 摄氏度，径向温差为 0.1169 摄氏度；含水率分别为 1.7662、1.0081，相差 0.7581。温度已接近均匀，水分仍有明显梯度。

图4在共同的时间与半径坐标上展示前三小时温湿场。温度由外向内升高，含水率由表面向内降低。升温逐渐接近完成时，水分迁移仍在继续，后续烘干主要受内部扩散及表层阻力影响。

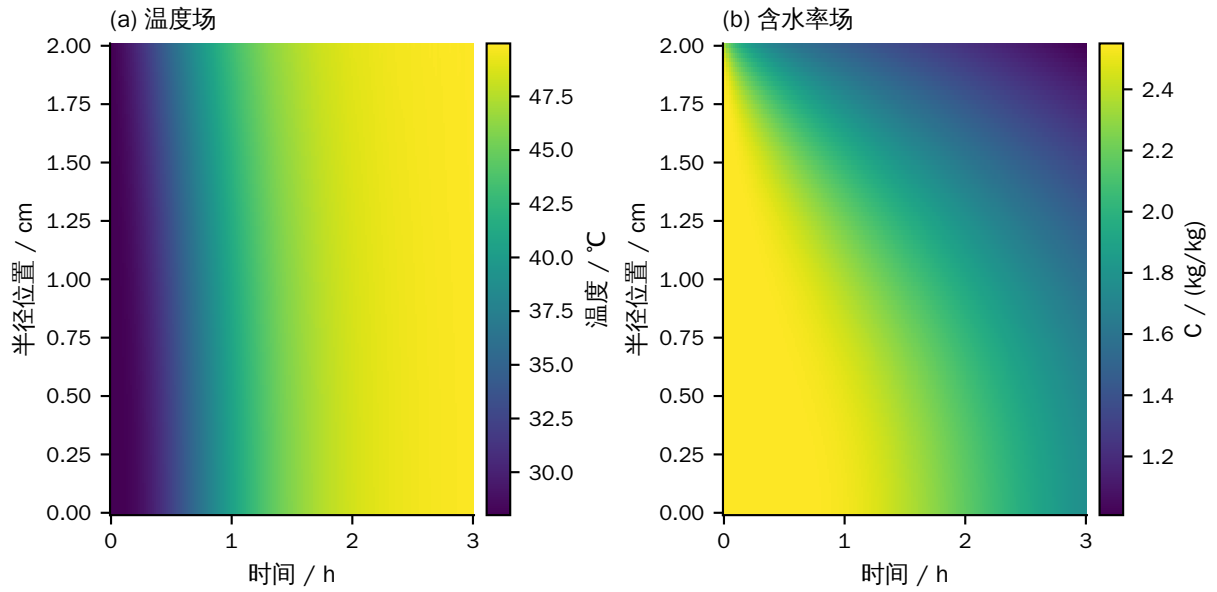


图 4: 问题二前三小时的温度与含水率场，按 60 秒间隔和 101 个径向重建点展示。

扩散系数随温度升高而增大，随含水率降低而减小。图5将求得的温湿场代入题设函数，展示两种作用共同形成的局部扩散系数变化，色标采用对数形式。

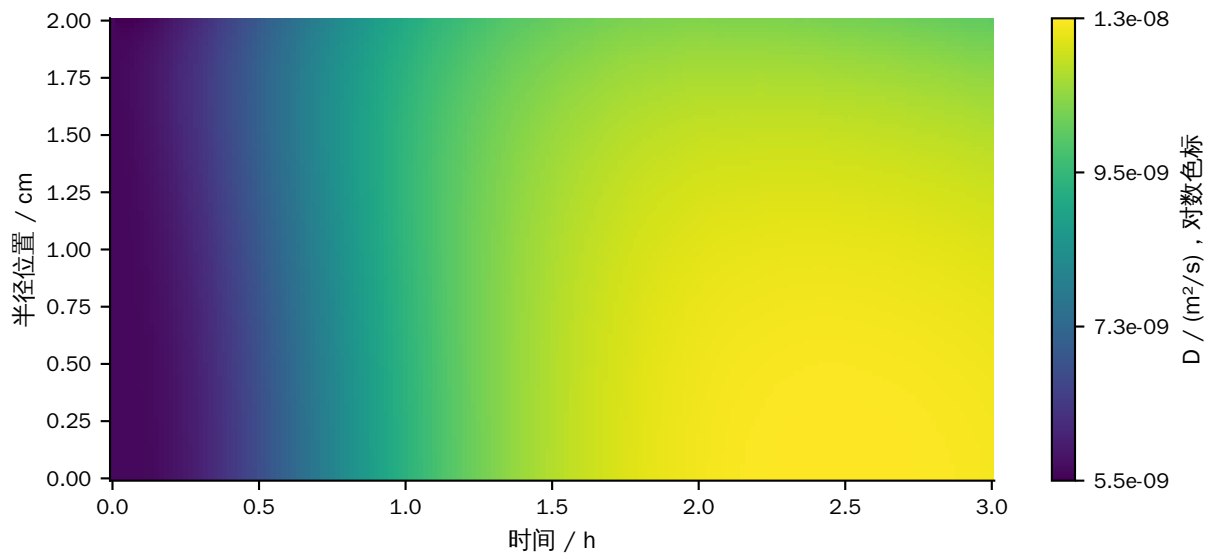


图 5: 问题二前三小时的局部扩散系数，由温湿解代入题设函数得到，色标采用对数形式。

7.6 模型检验

在 50 摄氏度下， $C = 2.55$ 时的扩散系数约为 1.3471×10^{-8} 平方米每秒； $C = 0.15$ 时为 8.0012×10^{-10} ， $C = 0.05$ 时降至 1.9833×10^{-12} 。即使表层已升温，含水率降低仍可使其成为低扩散区，这有助于解释后期脱水减慢。

含水率降低时，有效体积热容量和导热率均下降。前者使同等净热流引起的温度变化加快，后者减小内部导热能力；两种作用由联立方程共同确定。这些变化方向与图4及图5中的场分布一致，属于所用机理下的趋势检查。数值加密比较集中见第10章。

7.7 本问结论

在统一变物性与固定半径条件下，3 小时径向温差已降至 0.1169 摄氏度，中心和表面含水率仍为 1.7662、1.0081 kg/kg。水分均匀化明显滞后于升温，温度接近环境不足以作为完成指标；全过程含水率轨迹继续用于问题三。

8 问题三的模型建立与求解

8.1 建模思路

本问继承问题二的全场解，将求解目标由过程描述转为完成时间。用式(5)的径向最大含水率覆盖内部较慢脱水的位置，再以表面值和均值检查不同指标的判定差异。

8.2 变量与参数设置

固定半径 $R_0 = 2$ 厘米，物性采用式(13)，初态、传递系数和环境延拓与问题二相同。目标阈值为 0.15 kg/kg，搜索时间上限设为 7 天；若到达上限仍未越阈，则报告该区间内尚未完成。

8.3 模型建立

公共方程(2)和边界(4)保持不变，以式(6)定义的首次向下越阈事件作为终点。数值判据使用全部径向节点的未舍入含水率，保形重建避免在相邻节点间产生额外极值。

8.4 求解方法

沿用问题二的 640 区间网格与分段 BDF 积分，通过连续时间输出定位 $C_{\max} - 0.15$ 的零点，再积分至满足严格不等式的下一整秒。每 60 秒输出一次并补入终点，正文题表每 6 小时摘录；整秒记录与时空输出范围见附录C。事件的数值稳定性见第10章。

8.5 结果展示与分析

最终网格下，问题三的越阈时间为 57.4728 小时。终点表面含水率约为 0.0527，体积加权均值约为 0.1243。表9列出每 6 小时及严格终点的指定位置结果。

表 9: 问题三每 6 小时及严格终点的干基含水率（对应题目表 5）

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	2.0 cm
6	1.0170	0.9881	0.9015	0.7550	0.5328
12	0.4561	0.4432	0.4031	0.3297	0.1642
18	0.2988	0.2915	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1855	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
57.4728	0.1500	0.1477	0.1402	0.1249	0.0527

数值单位为 kg/kg。未行使用严格整秒终点，时间转为小时后显示四位小数；中心显示 0.1500 是舍入结果，严格不等式按未舍入值检查。

表9显示，中心含水率由 6 小时的 1.0170 降至 12 小时的 0.4561，48 小时仍为 0.1618，54 小时为 0.1539。接近阈值时，绝对剩余差值虽小，仍需较长时间继续脱水。这与低含水率区扩散系数迅速下降的特征一致。

图6给出终点径向剖面。中心附近接近阈值，表面接近且略高于有效边界值 0.05。表面与环境间的正含水率差维持向外传质，使内部水分继续排出。

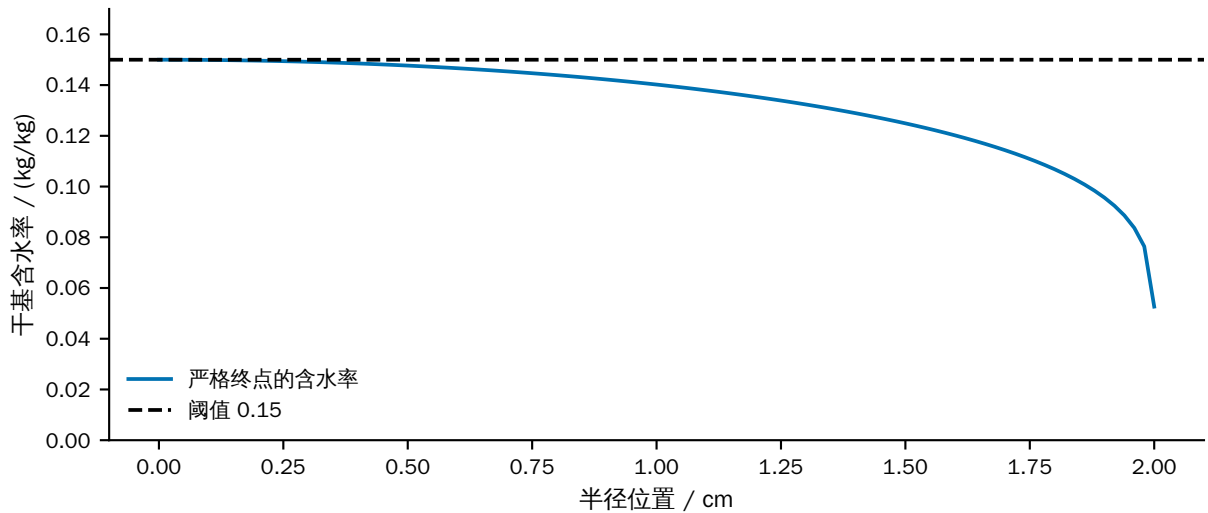


图 6: 问题三终点的径向含水率分布，中心接近阈值，表面含水率较低。

8.6 模型检验

图7比较三种含水指标。按 60 秒保存轨迹线性插值，表面值约在 12.5521 小时达到 0.15，均值约在 36.0251 小时达到 0.15；最大值的事件时间为 57.4728 小时。前两个时间

是轨迹插值近似值，最大值时间由事件求根得到。采用最大值判据能覆盖内部较慢脱水的位置。

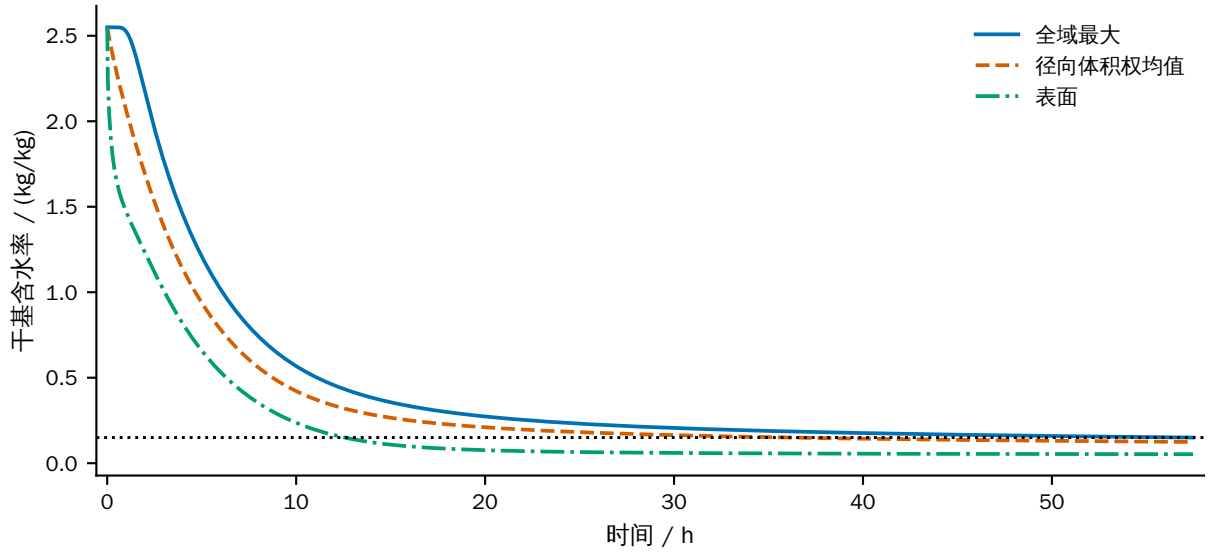


图 7: 问题三的最大、平均及表面含水率，按 60 秒间隔并补入终点；点线表示阈值 0.15。

该对照检验了完成指标与题意的一致性：表面值和均值越阈时，内部仍可有位置高于要求。已检查时刻的最大值位于中心附近，但计算始终对全部节点取最大值。

8.7 本问结论

在题给固定几何、物性及约定尾段环境下，最大含水率的越阈时间为 57.4728 小时。该时间回答一维径向场的各处达标要求；表面值和均值仅用于辅助描述，时间有效精度由第10章的加密结果限定。

9 问题四的模型建立与求解

9.1 建模思路

采用另一组物性和给定半径轨迹，在公共模型中补充材料运动。移动边界干燥可用 ALE 方法描述网格与材料输运 [4]，本问取网格随材料均匀径向收缩的情形。干质量守恒用于推导干基含水率方程，最大值事件仍按公共判据确定。

9.2 变量与参数设置

材料函数取原题附录 4 的式(15)，初态、长度、传递系数及尾段环境沿用公共设置。 $\rho_h(C)$ 用于有效热容量， $\rho_d(t)$ 由干质量与几何关系确定，两类密度按第4章的含义使用。

$$\begin{aligned}
\rho_h &= 760 + 90C, & c_p &= 1850 + \frac{2150C}{1+C}, \\
k &= 0.12 + \frac{0.20C}{1+C}, & D &= 0.00042 \exp\left(-\frac{0.30}{C} - \frac{3850}{T_K}\right).
\end{aligned} \tag{15}$$

半径按相邻记录线性插值, 见图8。6、12、24 小时的半径分别为 1.374、1.248、1.204 厘米, 末值为 1.198 厘米。收缩求解限定在 72 小时记录范围内。

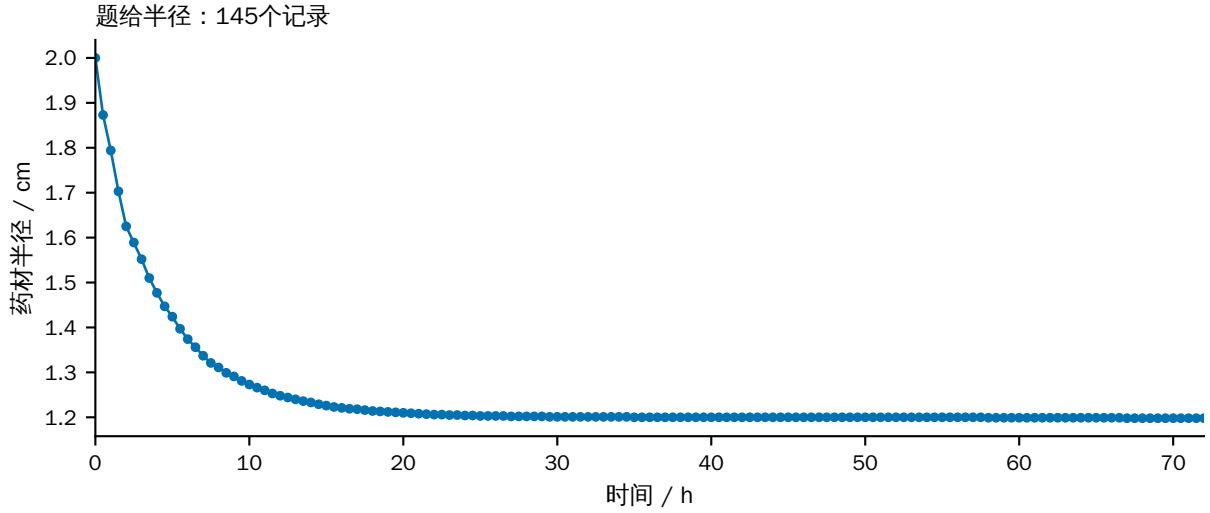


图 8: 附件 2 的 145 条半径记录及分段线性插值。

9.3 模型建立

材料速度为 $v = r\dot{R}/R$, 长度 L 保持不变; D/Dt 表示沿材料运动的物质导数。干质量 $M_d = \rho_d \pi R^2 L = \rho_{d0} \pi R_0^2 L$ 恒定, 水质量为 $M_w = M_d \bar{C}$ 。下面由两种质量的连续性关系推导含水率方程。均匀收缩使单位体积内两种质量同时变化, 压缩本身保持该比值。设干固体密度 $\rho_d(t)$ 在截面上均匀, 长度不变且干质量守恒, 则 $\rho_d R^2$ 为常数。取相对于固体的扩散通量为 $-\rho_d D \nabla C$, 由水分与干固体连续性方程得到式(16)。

$$\begin{aligned}
\frac{D\rho_d}{Dt} + \rho_d \nabla \cdot v &= 0, \\
\rho_d \frac{DC}{Dt} &= \nabla \cdot (\rho_d D \nabla C), \\
\rho_d(t) &= \rho_{d0} \left(\frac{R_0}{R(t)} \right)^2 \implies \frac{DC}{Dt} = \nabla \cdot (D \nabla C).
\end{aligned} \tag{16}$$

空间均匀的 $\rho_d(t)$ 可从散度中提出并约去, 得到式(18)中的水分方程, 压缩对两种质量的影响在干基比值中相消。若改用单位当前体积的水浓度, 压缩项的形式将随变量定义改变; 相关移动边界研究采用过这类体积变量 [5]。

令 $x = r/R(t)$, 对随材料输运的标量 u 定义 $U(x, t) = u(R(t)x, t)$, 可得式(17)。固定空间时间导数中的坐标变化项与材料平流项抵消, 随体坐标下的时间导数即为物质导数。

$$\left. \frac{\partial u}{\partial t} \right|_r = \left. \frac{\partial U}{\partial t} \right|_x - \frac{x\dot{R}}{R}U_x, \quad v u_r = \frac{x\dot{R}}{R}U_x, \quad \frac{Du}{Dt} = U_t. \quad (17)$$

固定 x 对应同一材料位置, 计算网格在归一化坐标中保持不变。

随体坐标下的方程和表面条件见式(18), 其中时间导数均在固定 x 下计算。沿用有限体积离散, 在各时刻更新半径和问题四物性。内部扩散含 $R(t)^{-2}$ 因子, 归一化表面交换含 $R(t)^{-1}$ 因子, 二者共同决定收缩后的传递速度。

$$\begin{aligned} A(C)\theta_t &= \frac{1}{R(t)^2 x} \partial_x (x k(C)\theta_x), \\ C_t &= \frac{1}{R(t)^2 x} \partial_x (x D(\theta, C)C_x), \\ -k_s \theta_x(1, t) &= R(t)h(\theta_s - \theta_\infty), \\ -D_s C_x(1, t) &= R(t)h_m(C_s - C_e). \end{aligned} \quad (18)$$

9.4 求解方法

在 $x \in [0, 1]$ 上沿用 640 区间有限体积网格和公共 BDF 设置, 各时刻以插值半径更新内部 $R(t)^{-2}$ 与表面 $R(t)^{-1}$ 因子。搜索上限为半径记录末端 72 小时, 遇到最大含水率越阈事件后补入严格整秒。固定物理位置通过 $x = r/R(t)$ 重建, $r > R(t)$ 的位置留空, 并单列实际表面值。

9.5 结果展示与分析

图9以物理半径为纵轴, 黑线表示插值半径 $R(t)$, 域外留白。固定物理半径对应水平线, 而同一材料点随 $r = xR(t)$ 运动, 应沿该轨迹读取其状态变化。

采用与问题三相同的最大含水率判据, 问题四的越阈时间为 51.0908 小时。终点表面含水率约为 0.0526, 体积权均值约为 0.1184, 半径为 1.2000 厘米。终点位于半径记录的 72 小时范围内。

表10列出每 6 小时及严格终点的指定位置含水率。在这些表列时刻, 0、0.5、1.0 厘米均位于药材内, 1.5 厘米均在域外。表面列对应实际 $R(t)$ 处状态, 随半径轨迹变化。完整输出中的域外位置按定义留空, 数量和规则见附录C。

问题四在 6 小时的中心含水率为 1.7196, 高于问题三的 1.0170; 42 小时为 0.1712, 略低于问题三的 0.1721。按 60 秒最大含水率轨迹插值, 两曲线约在 41.2 小时交叉。物性变化与扩散距离缩短共同影响全过程, 问题四在早期较湿, 最终却较早达到阈值。

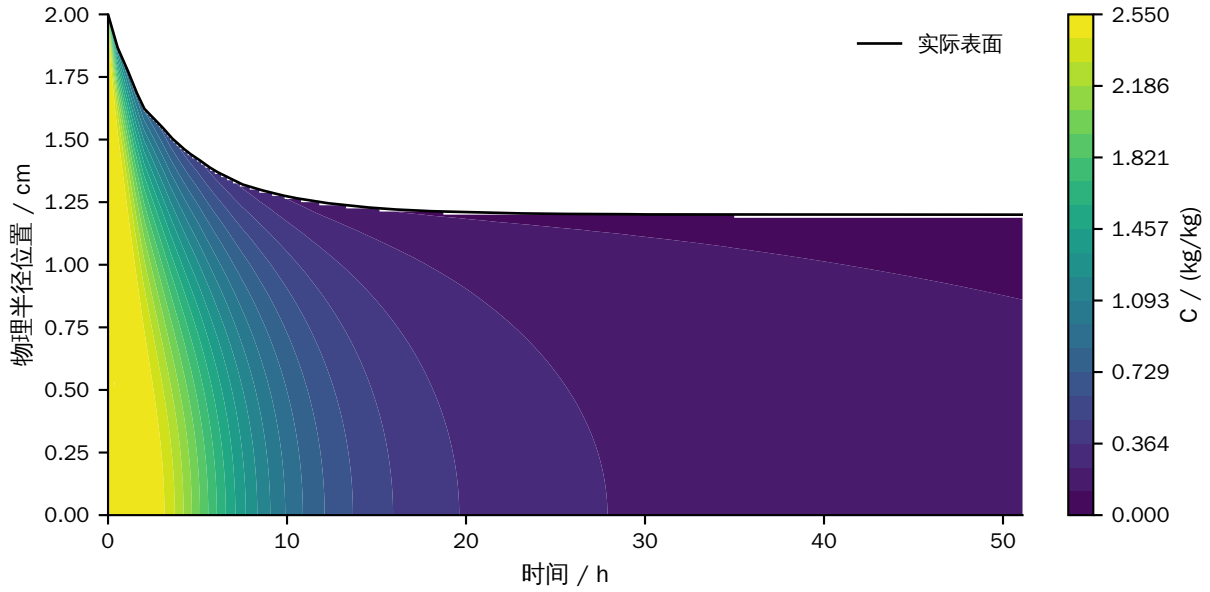


图 9: 问题四的移动域含水率场, 按 60 秒间隔并补入终点; 黑线为当前表面, 域外留白。

表 10: 问题四每 6 小时及严格终点的干基含水率 (对应题目表 6)

时间/h	0 cm	0.5 cm	1.0 cm	1.5 cm	移动表面
6	1.7196	1.5377	1.0226	—	0.4207
12	0.7377	0.6546	0.4083	—	0.1670
18	0.4087	0.3686	0.2397	—	0.0892
24	0.2851	0.2611	0.1790	—	0.0673
30	0.2263	0.2094	0.1493	—	0.0595
36	0.1928	0.1797	0.1317	—	0.0560
42	0.1712	0.1604	0.1200	—	0.0541
48	0.1561	0.1468	0.1116	—	0.0530
51.0908	0.1500	0.1413	0.1081	—	0.0526

“—”表示该时刻位置已在药材外, 结果工作簿对应单元格为空; 数值单位为 kg/kg。末行使用严格整秒终点, 时间转为小时后显示四位小数; 中心显示 0.1500 是舍入结果, 严格不等式按未舍入值检查。

9.6 模型检验

为考察给定物性下的收缩影响, 保持问题四的物性、初值、传递系数及环境延拓不变, 仅将半径固定为 2 厘米。对照越阈时间为 129.8449 小时, 长于收缩情景的 51.0908 小时。

记 $\tau(\mathcal{P}, R)$ 为物性组 $\mathcal{P}$ 和几何轨迹 R 下的越阈时间, $\mathcal{P}_3$ 、 $\mathcal{P}_4$ 分别表示问题三、四的整组物性。现有计算构成 $\tau(\mathcal{P}_3, R_0) \rightarrow \tau(\mathcal{P}_4, R_0) \rightarrow \tau(\mathcal{P}_4, R(t))$ 的比较路径。先改变物性使时间增加 72.3721 小时, 再引入收缩使时间减少 78.7541 小时, 净变化见式(19)。

$$\begin{aligned}
\Delta\tau_{\text{total}} &= \underbrace{\tau(\mathcal{P}_4, R_0) - \tau(\mathcal{P}_3, R_0)}_{\text{固定几何下的整组物性差异}} + \underbrace{\tau(\mathcal{P}_4, R(t)) - \tau(\mathcal{P}_4, R_0)}_{\text{问题四物性下的几何差异}} \\
&= 72.3721 - 78.7541 \approx -6.3819 \text{ h}.
\end{aligned} \tag{19}$$

差值由未舍入事件时间计算，展示时保留四位小数。沿这条路径，收缩引起的缩短超过整组物性改变引起的延长。该分解依赖所选比较路径，只描述整组物性与给定物性下几何变化的条件差异。

图10比较三种情景，式(20)分别计算总体降幅和同物性下的收缩降幅。前者以问题三为基准，后者以问题四物性、固定半径情景为基准，两个百分比具有不同分母。

$$\begin{aligned}
\eta_{\text{total}} &= 1 - \frac{\tau(\mathcal{P}_4, R(t))}{\tau(\mathcal{P}_3, R_0)} = 11.1043\%, \\
\eta_{\text{shrink}} &= 1 - \frac{\tau(\mathcal{P}_4, R(t))}{\tau(\mathcal{P}_4, R_0)} = 60.6524\%.
\end{aligned} \tag{20}$$

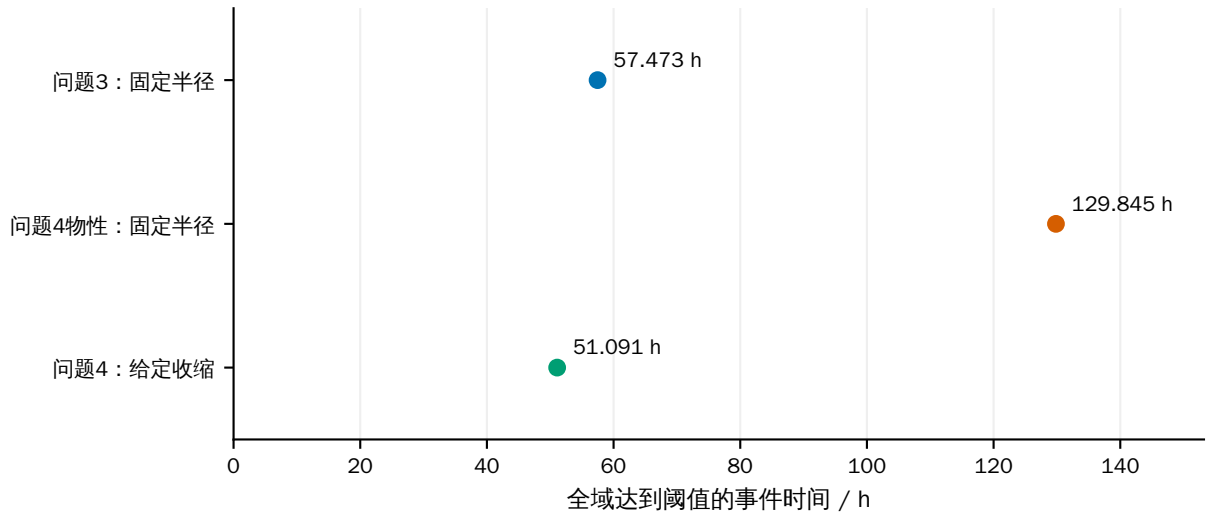


图 10: 三种情景的越阈时间：上、中两组几何相同，中、下两组物性相同。

问题四终点半径约为初始的 0.6 倍，半径平方约为初始的 0.36 倍，侧面积与体积之比约增至初始的 1.667 倍。这些变化分别影响内部扩散尺度和表面交换。实际半径与物性均随过程变化，完成时间由整个变化轨迹上的联立方程确定。

固定半径对照采用独立定义的恒定几何，即使计算超过 72 小时，也无需延拓收缩记录。该情景继续使用 4 小时后的恒定环境，用于比较给定物性下的几何影响。

9.7 本问结论

均匀径向收缩、长度不变及给定物性下，越阈时间为 51.0908 小时，终点半径为 1.2000 厘米。它比问题三早 6.3819 小时；同物性对照显示，所选比较路径上的几何作用超过整组物性变化造成的延长。该解释限定于给定收缩轨迹和有效参数。

10 综合检验

四问的局部检验回答结果如何符合题意及所用机理，本章集中检验离散格式、数值分辨率和模型内部关系。以下结果均来自现有解析算例、加密计算和情景比较。

10.1 误差与误差来源分析

误差主要来自三方面：空间与时间离散及结果重建造成数值差异；有限时段记录和边界延拓带来输入条件的不确定性；有效边界、潜热简化及空间运动假设限制预测范围。表11列出数值和内部一致性检验的参照对象，物理偏差需另有观测资料才能量化。

表 11: 验证项目、参照对象与可识别的错误

检验	参照对象	主要识别问题
固定域解析模态	Robin 边界 Bessel 解与精确越阈时间	柱坐标算子、中心和表面符号、事件定位
移动域非均匀模态	变半径零通量解析解	几何因子是否真正随时间更新
几何因子对照算例	只把 $R(t)^{-2}$ 冻结为 R_0^{-2}	固定几何因子对求解误差的影响
水分收支	离散均值与累计边界损失	面通量、积分权重和干基归一化
雅可比方向差分	实际右端的中心差分	状态及交叉导数是否一致
网格与容差复验	同一工况下更细离散结果	本题场与终点对数值设置的敏感性

采用 160、320 和 640 个区间逐级加密，在共同的物理位置与输出时刻比较。问题一覆盖前 1800 秒的全部逐秒点，问题二覆盖共同终端前的全过程逐秒点，问题三、四覆盖全部逐分钟点并加入共同终端。问题四比较实际域内位置及实际表面值，误差定义见式(21)。

$$E_\theta = \max_S |\theta_N - \theta_{2N}|, \quad E_C = \max_S |C_N - C_{2N}|, \quad E_t = \frac{|t_{*,N} - t_{*,2N}|}{t_{*,2N}}, \quad (21)$$

其中 S 为共同输出点集合。误差判据取 $E_\theta < 0.005$ 摄氏度、 $E_C < 5 \times 10^{-5}$ ；有终点事件的情景另要求 $E_t < 10^{-3}$ 。相邻网格之差用于估计数值设置的影响，结果在第10.3节集中比较。

10.2 灵敏度分析

末小时 61 条记录的温度均值为 49.998934 摄氏度、样本标准差 0.154353 摄氏度，水分记录均值为 0.04998754、样本标准差 0.00015013。均值接近尾段约定，但 4 小时以后仍属未观测工况；下列已计算情景用于比较其影响。

围绕两问的名义情景，分别采用末值延拓、末小时均值延拓、尾段温度加减 0.5 摄氏度、尾段有效边界含水率加减 0.001 以及传质系数加减 10%。环境延拓及温湿扰动从 4 小时后起作用，传质系数在全过程改变。每次只改变一项设置，越阈时间见表12。

表 12: 各单因素情景下的全域越阈时间, 单位: 小时

情景	问题三	问题四
名义环境及传质系数	57.4728	51.0908
四小时后采用末值	57.1698	50.8245
四小时后采用末小时均值	57.4745	51.0923
尾段温度降低 0.5 摄氏度	58.3959	51.9004
尾段温度升高 0.5 摄氏度	56.5700	50.2989
尾段有效边界值降低 0.001	57.4532	51.0691
尾段有效边界值升高 0.001	57.4931	51.1137
全过程传质系数降低 10%	58.2086	51.3820
全过程传质系数提高 10%	56.8954	50.8700

图11给出相对各自名义时间的变化。尾段温度降低 0.5 摄氏度时, 问题三、四的时间分别增加 1.6062%、1.5846%; 升高时分别减少 1.5707%、1.5500%。末小时均值延拓与名义情景的差异约为 0.003%, 末值延拓约缩短 0.52%。

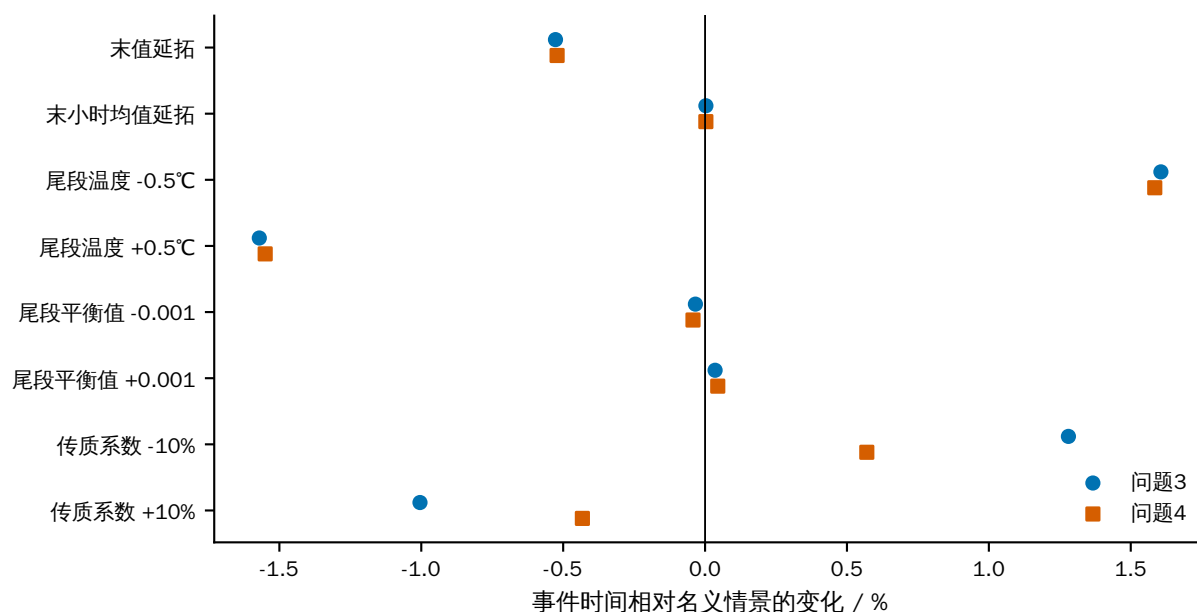


图 11: 各单因素情景相对本问名义时间的变化, 圆点与方块分别表示问题三和问题四。

传质系数降低 10% 时, 问题三、四的时间分别增加 1.2804%、0.5699%; 提高 10% 时, 分别减少 1.0046%、0.4323%。在所考察范围内, 外部传质条件仍影响完成时间。

各扰动的单位、相对幅度和作用时间不同, 结果用于描述所列情景的影响, 不能据此给出统一的参数重要性排序或统计置信区间。

在全部已计算的成对情景中, 问题四均早于问题三达到阈值, 该时间排序在所考察的局部变化下保持一致。

10.3 网格与时间步收敛性分析

表13列出两次加密的含水率差及最后一次加密的温度、事件差异。图12展示场差随网格加密的变化，比较范围与误差判据采用第10.1节定义的共同采样点。

表 13: 三网格比较; 末三列为 320 与 640 区间之差

问题	$E_C(160, 320)$	$E_C(320, 640)$	$E_\theta/\text{摄氏度}$	E_t
一	5.864×10^{-5}	1.468×10^{-5}	8.309×10^{-6}	—
二	5.130×10^{-5}	1.284×10^{-5}	1.444×10^{-5}	—
三	4.423×10^{-5}	1.106×10^{-5}	1.444×10^{-5}	2.434×10^{-5}
四	1.896×10^{-4}	4.742×10^{-5}	1.687×10^{-5}	1.511×10^{-5}

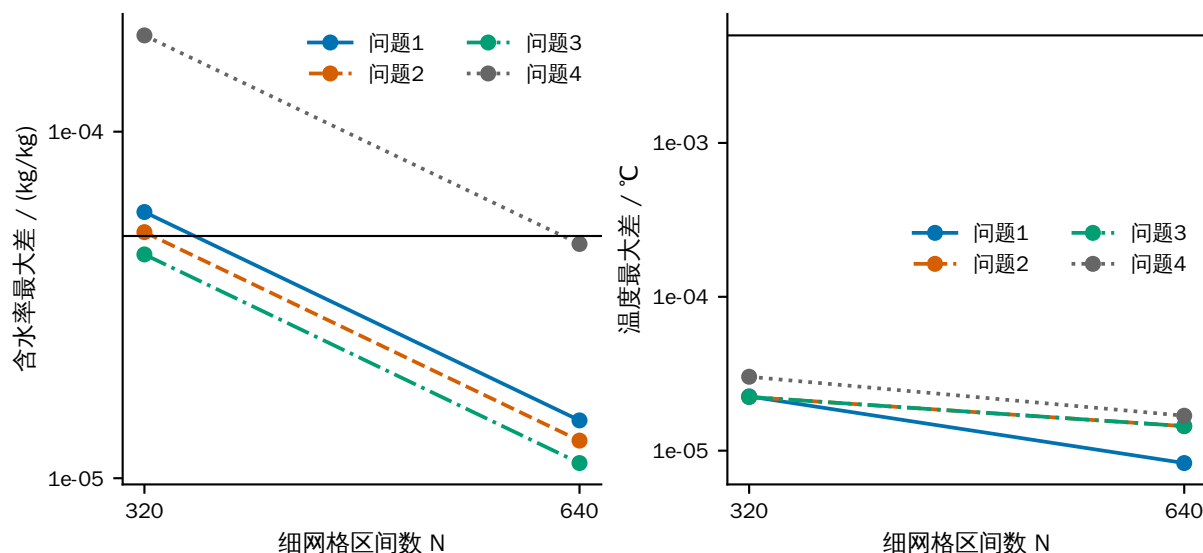


图 12: 相邻网格的最大场差, N 为细网格区间数; 横线为含水率与温度误差判据, 事件及时间精度结果见表13、14。

160 至 320 区间的比较中, 问题一、二、四的含水率差超过判据; 320 至 640 区间时, 全部指标满足要求, 故统一采用 640 区间。含水率差大体降为前一次的四分之一, 呈现接近二阶的收敛趋势。

问题三、四在 320 与 640 区间下的事件时间分别相差约 5.04 秒和 2.78 秒, 相对差均小于 10^{-3} 。整秒终点是当前离散模型按严格不等式给出的输出时刻, 现有网格比较尚未提供亚秒级绝对误差保证。

在 640 区间网格上, 将相对容差由 10^{-8} 收紧至 10^{-9} , 绝对容差相应收紧, 结果见表14。四问的最大温度差不超过 1.843×10^{-5} 摄氏度, 最大含水率差不超过 1.589×10^{-7} ; 问题三、四的事件相对差分别为 1.749×10^{-9} 和 3.510×10^{-9} 。

结合表13和表14, 含水率及终点时间对时间容差的敏感性小于对空间加密的敏感性。温度的两类差异均远低于 0.005 摄氏度的判据。上述比较支持当前数值设置下的场分布

表 14: 最终网格上两组时间容差的差异

问题	最大温度差/摄氏度	最大含水率差/(kg/kg)	事件相对差
一	8.635×10^{-6}	1.045×10^{-7}	—
二	1.476×10^{-5}	1.295×10^{-7}	—
三	1.476×10^{-5}	1.230×10^{-7}	1.749×10^{-9}
四	1.842×10^{-5}	1.589×10^{-7}	3.510×10^{-9}

与终点稳定性, 误差统计范围均为各检验定义的共同采样点集合。

10.4 稳健性与内部一致性检验

归一化水分收支 在空间均匀干固体密度条件下, 径向平均为 $\bar{C} = 2 \int_0^1 xC dx$ 。对水分方程积分并代入表面条件, 得到式(22)。令累计归一化损失 Λ 从零开始与温湿状态同步积分, 则 $\bar{C} + \Lambda$ 应保持初值 2.55。

$$\frac{d\bar{C}}{dt} = -\frac{2h_m}{R(t)}(C_s - C_e), \quad \dot{\Lambda} = \frac{2h_m}{R(t)}(C_s - C_e), \quad \varepsilon_M = \bar{C} + \Lambda - 2.55. \quad (22)$$

问题一、三按 1 秒间隔、问题四按 60 秒间隔检查, 均包含初始时刻和终点, 并使用全部求解节点。最大平衡残差分别约为 4.44×10^{-15} 、 4.88×10^{-15} 、 7.11×10^{-15} 。平均值和累计损失采用相同面通量构造, 这些残差用于检查离散通量、积分权重及归一化水分收支的一致性。

固定域解析解与导数一致性 对常扩散系数 $d_0 = 10^{-5}$ 平方米每秒、半径 0.02 米、 $C_e = 0.05$ 、 $h_m = d_0/R$ 的算例, 取式(23)的 Bessel 模态初值。其中 β 为 $\beta J_1(\beta) = J_0(\beta)$ 的第一正根, 解析解同时满足中心对称和 Robin 表面条件。该组参数专用于数值验证。

$$C(r, t) = 0.05 + 0.20 J_0\left(\beta \frac{r}{R}\right) \exp\left(-\frac{d_0 \beta^2}{R^2} t\right), \quad t_{\text{exact}} = \frac{R^2 \log 2}{d_0 \beta^2}. \quad (23)$$

采用 160 个区间时, 精确越阈时间为 17.581493358 秒, 数值事件时间为 17.581482896 秒, 相差约 1.05×10^{-5} 秒。该算例共同检验柱坐标离散、中心处理、表面通量符号和事件定位。另取零传质系数与均匀初值, 离散右端为零, 封闭均匀场保持不变。

以方向差分比较 $J(y)p$ 和 $[F(y + \varepsilon p) - F(y - \varepsilon p)]/(2\varepsilon)$, 检查解析雅可比与方程右端的一致性。检验覆盖四问, 并在常湿、低湿、均匀及加密网格下比较 16 组块导数, 最大相对差约为 3.78×10^{-10} 。

移动域非均匀解析基准 为检验变化半径下的扩散速度, 取 $R(t) = R_0(1 - \gamma t)$, 其中 $R_0 = 0.02$ 米、 $\gamma = 1/7200$ 秒⁻¹、 $0 \leq t \leq 3600$ 秒, 半径由 0.02 米缩至 0.01 米。扩散

系数固定为 $D_0 = 10^{-8}$ 平方米每秒，中心和表面均为零通量。取 $J_1(\lambda) = 0$ 的第一正根 $\lambda = 3.8317059702$ ，式(24)给出非均匀初值及其解析解。

$$\begin{aligned} C(x, t) &= 1 + 0.3J_0(\lambda x) \exp[-\lambda^2 D_0 I(t)], \\ I(t) &= \int_0^t \frac{ds}{R(s)^2} = \frac{t}{R_0^2(1 - \gamma t)}. \end{aligned} \quad (24)$$

由 $J'_0(z) = -J_1(z)$ 可知，该解在 $x = 0, 1$ 处均满足零梯度，Bessel 特征方程保证其满足常系数水分方程。空间非均匀模态按 $I(t)$ 衰减，能检验几何因子对扩散速度的影响。算例使用本文采用的水分离散算子，仅替换物性、半径轨迹和边界条件。

空间加密时，相对容差取 10^{-11} ，最大时间步取 14.0625 秒，在 0 至 3600 秒的 129 个等间隔时刻、各网格全部节点上比较数值解和解析解。表15给出最大绝对误差。随着区间数增加，误差逐级下降，640 区间时为 8.11×10^{-7} ，低于 10^{-5} 的误差判据。

表 15: 移动域解析算例的空间加密与几何因子对照

算子	径向区间数	最大解析对比误差/(kg/kg)
随时间更新 $R(t)^{-2}$	40	2.05124×10^{-4}
随时间更新 $R(t)^{-2}$	80	5.17059×10^{-5}
随时间更新 $R(t)^{-2}$	160	1.29592×10^{-5}
随时间更新 $R(t)^{-2}$	320	3.24227×10^{-6}
随时间更新 $R(t)^{-2}$	640	8.10753×10^{-7}
故意冻结为 R_0^{-2}	640	5.86815×10^{-2}

时间加密采用 160 区间均匀网格，相对容差依次为 10^{-3} 、 10^{-5} 、 10^{-7} ，最大时间步依次为 450、225、112.5 秒。以同一半离散矩阵 L_N 的指数解 $\exp[D_0 I(t) L_N] C(0)$ 为参照，时间误差依次为 1.60038×10^{-3} 、 2.56224×10^{-5} 、 2.65945×10^{-7} ，最细设置满足 5×10^{-6} 的误差判据。该设置与连续 Bessel 解的总误差为 6.37797×10^{-6} ，其中包含固定空间网格的离散误差。

对照算例保持相同的移动半径、非均匀初值和零通量边界，仅将离散方程中的 $R(t)^{-2}$ 固定为 R_0^{-2} 。误差增至约 5.87×10^{-2} ，明显超过 10^{-5} 的判据。该情形仍保持离散水量，表明几何因子的正确性需结合空间分布比较判断。

正确离散下，平均含水率相对初始离散加权均值的漂移不超过 6.66×10^{-16} 。此处比较同一离散求积下的前后变化，与初值相对连续均值 1 的求积误差分开。固定域、收缩域的水分收支及无通量均匀场结果与上述检查一致。

10.5 验证结论及其适用边界

解析算例及加密比较检查离散格式、积分精度和终点稳定性；水分平衡与量纲核对检查模型内部关系。两类证据共同提高所用方程数值结果的可信度。附件没有提供内部温湿剖面或独立失水量观测，尚未开展实际药材的物理验证。

已有敏感性计算仅覆盖尾段环境与传质系数的指定扰动，未识别吸附等温线，也未量化潜热、轴向差异及非均匀收缩的影响。结论据此限定在声明的模型与已计算情景内。

11 模型评价与推广

11.1 模型优点

统一的干基变量、面通量离散和最大值判据使四问衔接清楚。固定域与均匀收缩域共享求解框架，保留有限表面交换阻力、低湿表层阻力和内部滞后。三情景对照将总体时间差与给定物性下的几何影响分开，解析基准及加密比较则提供了数值依据。

11.2 模型不足

主要不确定性来自表2所列的有效边界、未观测尾段环境和空间运动近似。现有局部敏感性只覆盖部分输入条件，尚未量化这些结构性假设的影响。初始干质量尺度未给定，质量分析以含水率和归一化失水量为主。

11.3 模型适用范围

当前结果对应题给物性、初态、传递系数和约定环境，可用于比较轴向、周向近似均匀圆柱体的径向温湿分布及阈值时间。问题四还要求内部运动可由给定半径下的均匀径向收缩近似，并将计算限制在半径记录范围内。数值稳定性与物理预测范围的区别见第10章。

11.4 模型改进与推广

对于具有相同空间对称性的其他圆柱对象，可替换 R_0 、物性函数、环境记录及表面传递系数，沿用现有离散方法重新求解。给定新的均匀收缩轨迹时，可沿用随体坐标处理；若边界或运动显著不均匀，则需相应空间数据与扩展后的控制方程。

后续资料宜围绕已有缺口收集：吸附等温线用于约束有效边界，内部温湿观测用于区分模型偏差与数值误差，局部尺寸记录用于检查均匀收缩。现阶段保留已声明的理想模型，不预设未经计算的改进幅度。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于题意分析、文献线索整理、程序设计与核验、图表制作和文稿编辑，详细使用情况见支撑材料《AI 工具使用详情》。

所用工具为 OpenAI Codex。本轮由 AI 将公共模型与各问求解分章整理，编辑摘要和论证文字，核对既有结果并编译检查；未重新计算模型，核心方程和数值结论保留。参赛队最终人工审阅尚待完成，使用者须依据真实记录确认披露，独立理解和核对方程、程序、结果与引用，并对提交内容负责。

参考文献

- [1] da Silva W. P., e Silva C. M. D. P. S., Gomes J. P. Comparison of boundary conditions to describe drying of turmeric (*Curcuma longa*) rhizomes using diffusion models[J]. *Journal of Food Science and Technology*, 2014, 51(11): 3181–3189. DOI: 10.1007/s13197-012-0813-x.
- [2] da Silva W. P., e Silva C. M. D. P. S., Gama F. J. A. Estimation of thermo-physical properties of products with cylindrical shape during drying: The coupling between mass and heat[J]. *Journal of Food Engineering*, 2014, 141: 65–73. DOI: 10.1016/j.jfoodeng.2014.05.010.
- [3] Tzempelikos D. A., Mitrakos D., Vouros A. P., Bardakas A. V., Filios A. E., Margaritis D. P. Numerical modeling of heat and mass transfer during convective drying of cylindrical quince slices[J]. *Journal of Food Engineering*, 2015, 156: 10–21. DOI: 10.1016/j.jfoodeng.2015.01.017.
- [4] Seyedabadi E., Khojastehpour M., Abbaspour-Fard M. H. Convective drying simulation of banana slabs considering non-isotropic shrinkage using FEM with the Arbitrary Lagrangian–Eulerian method[J]. *International Journal of Food Properties*, 2017, 20(sup1): S36–S49. DOI: 10.1080/10942912.2017.1288134.
- [5] Adrover A., Venditti C., Brasiello A. A Non-Isothermal Moving-Boundary Model for Continuous and Intermittent Drying of Pears[J]. *Foods*, 2020, 9(11): 1577. DOI: 10.3390/foods9111577.

A 支撑材料文件清单

表16列出支撑材料文件及用途。附录 B 给出七个脚本、五个辅助模块及依赖清单，与电子支撑材料中的对应源文件一致。运行时保留附件、结果模板和模块的相对目录。

表 16: 电子支撑材料目录

文件或目录	内容
原题及附件目录	题目 PDF、环境、半径及四个原始结果模板
<code>solve.py</code>	物性、网格、通量、雅可比、积分、事件与小例检验
<code>run_all.py</code>	网格和时间复验、结果表、场数组及情景分析
<code>make_figures.py</code>	从真实结果生成 14 组矢量和位图
<code>verify_moving_domain.py</code>	移动域解析模态、时空加密、几何因子对照与守恒检验
<code>export_result2_full.py</code>	全过程逐秒工作簿的分块导出和完整结构检查
<code>reproduce.py</code>	完整计算、验证和导出的运行入口
<code>test_sync.py</code>	采样统计与 600 秒导出的短程检查
<code>utils/</code>	全量读表、绘图风格、图表自检、导出与环境记录
<code>requirements.txt</code>	原计算使用的 Python 包版本
<code>results/</code>	四个 XLSX、六张题表、过程数组、诊断与参数摘要
<code>figures/</code>	原始输入、运行过程和结果图的 PDF、SVG、PNG
完整论文 LaTeX 目录	主文稿、章节、表格、图形及完整代码副本
AI 工具使用详情 PDF	关键提示、采纳范围、工具说明和人工复核提示

计算环境为 Python 3.10.12，包版本见 `requirements.txt`。准备依赖及文泉驿正黑字体 `wqy-zenhei.ttc` 后，从包含原始附件的项目根目录执行：

```
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 -B reproduce.py
```

完整入口在 `reproduced_*` 目录依次计算解析基准、四问结果、空间与时间精度比较、工作簿、情景对照和图形，保留原有结果。需要短程检查或单独运行移动域算例时，可分别使用：

```
python3 -B reproduce.py --smoke
python3 -B verify_moving_domain.py
python3 -B verify_moving_domain.py --mutant-only
```

短程检查覆盖 600 秒；最后一条只计算冻结几何因子的对照，其误差应超过正常算例的判据。正式问题二文件仍按全过程逐秒输出。

论文在 LaTeX 工程内以 XeLaTeX 编译两遍，宏包与字体要求见支撑材料说明。支撑包含全部程序、结果、图形、LaTeX 源码及 AI 使用详情，主论文 PDF 单独提供。字体二进制未重复打包，使用者须按字体许可准备。

B 完整可运行源代码

B.1 核心求解与诊断

```
"""Nonlinear radial heat/moisture diffusion, SI units throughout.

Run from the project directory: python3 solve.py --smoke
All assumptions and boundary extrapolations are defined in the model report.
"""

from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
import argparse
import hashlib
import importlib.util
import json
import math
import sys
import time

ROOT = Path(__file__).resolve().parent
if (ROOT / '.work/deps').exists():
    sys.path.insert(0, str(ROOT / '.work/deps'))
import numpy as np
from scipy.integrate import solve_ivp
from scipy.interpolate import PchipInterpolator
from scipy.optimize import brentq
from scipy.sparse import bmat, csc_matrix, coo_matrix, diags
from scipy.special import j0, j1

SEED = 20260910

def read_inputs():
    from utils.read_rows import read_excel_rows
    output, snapshots = [], []
    for name, rows, first, last in (
        ('附件 1.xlsx', 241, [0,28,.01963], [14400,50.165,.04986]),
        ('附件 2.xlsx', 145, [0,2], [259200,1.198])):
        path = ROOT / '附件' / name
        data = np.array([list(r.values()) for r in read_excel_rows(path, header=True,
            expected_rows=rows)], float)
        assert np.allclose(data[0], first) and np.allclose(data[-1], last)
        assert np.all(np.isfinite(data)) and np.all(np.diff(data[:,0]) > 0)
        output.append(data)
        snapshots.append(dict(path=str(path.relative_to(ROOT)),
            sha256=hashlib.sha256(path.read_bytes()).hexdigest(),
            rows=rows, first=data[0].tolist(), last=data[-1].tolist()))
    assert np.all(np.diff(output[1][:,1]) <= 0)
    return output, snapshots

def properties(q, temperature, moisture):
    if np.any(moisture <= 0) or np.any(temperature <= -273.15):
        raise ValueError('Nonpositive moisture or absolute temperature')
    if q == 1:
        return np.full_like(moisture, 820*2600), np.full_like(moisture,.36), 7e-9*np.exp(-.89/moisture)
    f = moisture/(1+moisture)
```

```

if q in (2,3):
    capacity = (650+128*moisture)*(1450+2736*f)
    return capacity, .21+.38*f, .0024*np.exp(-.45/moisture-3850/(temperature+273.15))
if q == 4:
    capacity = (760+90*moisture)*(1850+2150*f)
    return capacity, .12+.20*f, .00042*np.exp(-.30/moisture-3850/(temperature+273.15))
raise ValueError(q)

class Grid:
    def __init__(self, n, graded=True):
        self.n = n
        s = np.linspace(0.,1.,n+1)
        self.x = 1-(1-s)**2 if graded else s
        self.dx = np.diff(self.x)
        self.faces = np.r_[0., (self.x[:-1]+self.x[1:])/2, 1.]
        self.w = np.diff(self.faces**2)/2
        block = diags([np.ones(n),np.ones(n+1),np.ones(n)],[-1,0,1],format='csc')
        self.scalar_pattern = block
        base = bmat([[block,block],[block,block]],format='csc')
        last = csc_matrix([1.],[0],[2*n+1]), shape=(1,2*(n+1))
        self.pattern = bmat([[base,csc_matrix((2*(n+1),1))],[last,csc_matrix((1,1))]],format='csc')

    def rate(self, u, a, radius, transfer, ambient, capacity=1.):
        a = np.broadcast_to(a, u.shape)
        af = 2*a[:-1]*a[1:]/(a[:-1]+a[1:])
        flux = np.r_[0.,self.faces[1:-1]*af*np.diff(u)/self.dx,radius*transfer*(ambient-u[-1])]
        return np.diff(flux)/(radius**2*self.w*capacity)

    def jac_block(self, u, a, da, radius, transfer, capacity=1., same=False):
        """Derivative of conservative fluxes with respect to one state field."""
        m = self.n+1
        a = np.broadcast_to(a, u.shape)
        da = np.broadcast_to(da, u.shape)
        den = radius**2*self.w*capacity
        left, right = a[:-1], a[1:]
        harmonic = 2*left*right/(left+right)
        fac = self.faces[1:-1]/self.dx
        delta = np.diff(u)
        dl = fac*(2*right**2/(left+right)**2*da[:-1]*delta
            - (harmonic if same else 0.))
        dr = fac*(2*left**2/(left+right)**2*da[1:]*delta
            + (harmonic if same else 0.))
        i = np.arange(m-1)
        row = np.r_[i,i,i+1,i+1]
        col = np.r_[i,i+1,i,i+1]
        val = np.r_[dl/den[:-1],dr/den[:-1],-dl/den[1:],-dr/den[1:]]
        if same:
            row = np.r_[row,m-1]; col = np.r_[col,m-1]
            val = np.r_[val,-radius*transfer/den[-1]]
        return coo_matrix((val,(row,col)),shape=(m,m)).tocsc()

@dataclass
class Scenario:
    tail: str = 'nominal'
    temperature_shift: float = 0.
    moisture_shift: float = 0.

```

```

mass_factor: float = 1.
fixed_radius: bool = False

class Model:
    def __init__(self, q, n, arrays, scenario=None, graded=True):
        self.q, self.grid = q, Grid(n,graded)
        self.boundary, self.shrink = arrays
        self.scenario = scenario or Scenario()

    def radius(self,t):
        if self.q != 4 or self.scenario.fixed_radius:
            return np.asarray(t)*0+.02
        if np.max(t) > self.shrink[-1,0]+1e-7:
            raise ValueError('Radius extrapolation beyond measured 72 h is not authorised')
        return np.interp(t,self.shrink[:,0],self.shrink[:,1])/100

    def ambient(self,t):
        b, s = self.boundary,self.scenario
        if t <= b[-1,0]:
            return np.interp(t,b[:,0],b[:,1]),np.interp(t,b[:,0],b[:,2])
        tail = b[-1,1:] if s.tail=='last' else b[b[:,0]>=10800,1:].mean(axis=0) if s.tail=='mean' else
        (50.,.05)
        return tail[0]+s.temperature_shift,tail[1]+s.moisture_shift

    def rhs(self,t,y):
        m = self.grid.n+1
        temp, moisture = y[:m],y[m:2*m]
        cap,k,d = properties(self.q,temp,moisture)
        radius = float(self.radius(t))
        ti,ce = self.ambient(t)
        hm = 8e-7*self.scenario.mass_factor
        tr = self.grid.rate(temp,k,radius,25.,ti,cap)
        cr = self.grid.rate(moisture,d,radius,hm,ce)
        loss_rate = 2*hm*(moisture[-1]-ce)/radius
        return np.r_[tr,cr,loss_rate]

    def jac(self,t,y):
        m = self.grid.n+1
        temp, c = y[:m], y[m:2*m]
        cap,k,d = properties(self.q,temp,c)
        if self.q == 1:
            dc = np.zeros(m); kc = np.zeros(m); dt = np.zeros(m)
            dmoist = d*.89/c**2
        elif self.q in (2,3):
            rho=650+128*c; cp=1450+2736*c/(1+c)
            dc=128*cp+rho*2736/(1+c)**2
            kc=.38/(1+c)**2; dt=d*3850/(temp+273.15)**2
            dmoist=d*.45/c**2
        else:
            rho=760+90*c; cp=1850+2150*c/(1+c)
            dc=90*cp+rho*2150/(1+c)**2
            kc=.20/(1+c)**2; dt=d*3850/(temp+273.15)**2
            dmoist=d*.30/c**2
        radius=float(self.radius(t)); ti,ce=self.ambient(t)
        hm=8e-7*self.scenario.mass_factor; g=self.grid
        tt=g.jac_block(temp,k,0.,radius,25.,cap,same=True)

```

```

tc=g.jac_block(temp,k,kc,radius,25.,cap)
tr=g.rate(temp,k,radius,25.,ti,cap)
tc=tc-diags(tr*dc/cap,format='csc')
cc=g.jac_block(c,d,dmoist,radius,hm,same=True)
ct=g.jac_block(c,d,dt,radius,hm)
loss=csc_matrix((([2*hm/radius],[0],[m-1])),shape=(1,m))
return bmat([[tt,tc,csc_matrix((m,1))],
             [ct,cc,csc_matrix((m,1))],
             [csc_matrix((1,m)),loss,csc_matrix((1,1))]],format='csc')

def event(self,t,y):
    m = self.grid.n+1
    return np.max(y[m:2*m])-.15

class Run:
    def __init__(self,model,segments,crossing,strict,runtime):
        self.model,self.segments = model,segments
        self.crossing,self.strict,self.runtime = crossing,strict,runtime
        self.end = float(segments[-1].t[-1])

    def values(self,times):
        times = np.atleast_1d(times).astype(float)
        if times.min() < 0 or times.max() > self.end+1e-6:
            raise ValueError('Requested sampling outside integrated interval')
        y = np.empty((2*(self.model.grid.n+1)+1,len(times)))
        assigned = np.zeros(len(times),bool)
        for segment in self.segments:
            take=(times>=segment.t[0]-1e-8)&(times<=segment.t[-1]+1e-8)
            if not np.any(take):
                continue
            y[:,take]=segment.sol(times[take])
            assigned[take]=True
        assert assigned.all()
        return y

    def sample(self,times,radii_cm=None,normalized=False):
        times = np.atleast_1d(times).astype(float)
        y=self.values(times); m=self.model.grid.n+1
        if radii_cm is None:
            radii_cm=np.linspace(0,2,21)
        coords=np.asarray(radii_cm,dtype=float)
        temperatures,moistures=[],[]
        for j,t in enumerate(times):
            x=coords if normalized else coords/100/float(self.model.radius(t))
            temperatures.append(PchipInterpolator(self.model.grid.x,y[:m,j],extrapolate=False)(x))
            moistures.append(PchipInterpolator(self.model.grid.x,y[m:2*m,j],extrapolate=False)(x))
        return np.array(temperatures),np.array(moistures)

    def summary(self, sample_step_s=None, chunk_size=1024):
        """Diagnostics on an explicit sample grid, never continuous extrema.

        Default: 250 uniformly spaced times including both endpoints.
        A positive sample_step_s uses 0, step, ..., plus the exact endpoint.
        All radial solver nodes participate; chunking bounds memory use.
        """
        if chunk_size < 1 or int(chunk_size) != chunk_size:

```

```

        raise ValueError('chunk_size must be a positive integer')
    if sample_step_s is None:
        times=np.unique(np.r_[0.,np.linspace(0,self.end,250),self.end])
    else:
        if not np.isfinite(sample_step_s) or sample_step_s <= 0:
            raise ValueError('sample_step_s must be finite and positive')
        times=np.unique(np.r_[np.arange(0,self.end,sample_step_s),self.end])
    m=self.model.grid.n+1
    tmin=cmin=float('inf'); tmax=cmax=-float('inf')
    balance=above=0.
    for start in range(0,len(times),int(chunk_size)):
        y=self.values(times[start:start+int(chunk_size)])
        t,c=y[:m],y[m:2*m]
        if not np.isfinite(y).all():
            raise ValueError('Nonfinite state on diagnostic sample grid')
        means=2*self.model.grid.w@c
        balance=max(balance,float(np.abs(means+y[-1]-2.55).max()))
        above=max(above,float((c.max(axis=0)-c[0]).max()))
        tmin=min(tmin,float(t.min())); tmax=max(tmax,float(t.max()))
        cmin=min(cmin,float(c.min())); cmax=max(cmax,float(c.max()))
    yend=self.values([self.end]); t,c=yend[:m],yend[m:2*m]
    return dict(question=self.model.q,n=self.model.grid.n,end_s=self.end,
                crossing_s=self.crossing,strict_s=self.strict,
                crossing_h=None if self.crossing is None else self.crossing/3600,
                strict_h=None if self.strict is None else self.strict/3600,
                end_radius_cm=float(self.model.radius(self.end))*100,
                T_center_end=float(t[0,-1]),T_surface_end=float(t[-1,-1]),
                C_center_end=float(c[0,-1]),C_surface_end=float(c[-1,-1]),Cmax_end=float(c[:,-1].max()),
                diagnostic_sampling=dict(
                    time_grid='uniform_250_including_endpoints' if sample_step_s is None else
                    'fixed_step_plus_endpoint',
                    time_count=len(times),first_time_s=float(times[0]),last_time_s=float(times[-1]),
                    step_s=float(times[1]-times[0]) if sample_step_s is None and len(times)>1 else
                    sample_step_s,
                    radial_grid='all_solver_nodes',radial_node_count=m,
                    continuous_extrema_claimed=False),
                moisture_balance_max_sampled=balance,
                max_above_center_sampled=above,
                T_min_sampled=tmin,T_max_sampled=tmax,C_min_sampled=cmin,C_max_sampled=cmax,
                nfev=sum(s.nfev for s in self.segments),runtime_s=self.runtime)

def integrate(q,n,arrays,horizon=None,stop_at_threshold=None,scenario=None,rtol=1e-8,graded=True):
    start=time.perf_counter()
    model=Model(q,n,arrays,scenario,graded)
    if horizon is None:
        horizon={1:1800.,2:10800.,3:604800.,4:259200.}[q]
    stop_at_threshold=q in (3,4) if stop_at_threshold is None else stop_at_threshold
    def event(t,y): return model.event(t,y)
    event.terminal=True;event.direction=-1
    m=n+1; y=np.r_[np.full(m,28.),np.full(m,2.55),0.]
    atol=np.r_[np.full(m,rtol),np.full(m,rtol*.01),rtol*.01]
    segments=[];crossing=None;strict=None;begin=0.
    for end in sorted(set([min(horizon,14400.),float(horizon)])):
        if end<=begin: continue
        sol=solve_ivp(model.rhs,(begin,end),y,method='BDF',rtol=rtol,atol=atol,

```

```

        jac=model.jac,dense_output=True,
        events=event if stop_at_threshold else None,max_step=15. if begin<14400 else 600.)
    if not sol.success: raise RuntimeError(sol.message)
    segments.append(sol);y=sol.y[:,-1];begin=float(sol.t[-1])
    if sol.status==1:
        crossing=float(sol.t_events[0][0]);strict=math.floor(crossing)+1
        extra=solve_ivp(model.rhs,(crossing,strict),y,method='BDF',rtol=rtol,atol=atol,
            jac=model.jac,dense_output=True)
        if not extra.success or model.event(strict,extra.y[:,-1])>=0:
            raise RuntimeError('Strict completion endpoint failed')
        segments.append(extra)
        break
    run=Run(model,segments,crossing,strict,time.perf_counter()-start)
    info=run.summary()
    assert info['C_min_sampled']>0 and info['C_max_sampled']<=2.5500001
    assert info['T_min_sampled']>=27.99999 and info['T_max_sampled']<=max(arrays[0][:,1].max(),50.5)+1e-5
    assert info['moisture_balance_max_sampled']<1e-7
    return run

def smoke():
    arrays,snapshots=read_inputs()
    checks=[]
    for q in (1,2,4):
        run=integrate(q,80,arrays,horizon=600,stop_at_threshold=False)
        temp,moist=run.sample([1,60,600])
        checks.append(dict(**run.summary(),sample_T=np.where(np.isfinite(temp),temp,None).tolist(),sample_
            _C=np.where(np.isfinite(moist),moist,None).tolist()))

    jac_errors=[]
    rng=np.random.default_rng(SEED)
    for q in (1,2,3,4):
        model=Model(q,24,arrays)
        m=25; x=model.grid.x
        y=np.r_[32+3*x**2,2.3-.4*x**2,0.]
        direction=np.r_[rng.normal(size=m),.1*rng.normal(size=m),0.]
        eps=1e-5
        fd=(model.rhs(3600.,y+eps*direction)-model.rhs(3600.,y-eps*direction))/(2*eps)
        exact_jv=model.jac(3600.,y)@direction
        error=float(np.linalg.norm(fd-exact_jv,np.inf)/max(np.linalg.norm(exact_jv,np.inf),1e-30))
        assert error<1e-6
        jac_errors.append(dict(question=q,relative_directional_error=error))
    grid=Grid(160)
    beta=brentq(lambda b:b*j1(b)-j0(b),1,2)
    exact=math.log(2)*.02**2/(1e-5*beta**2)
    rhs=lambda t,c:grid.rate(c,1e-5,.02,1e-5/.02,.05)
    def event(t,c):return c.max()-.15
    event.terminal=True;event.direction=-1
    result=solve_ivp(rhs,(0,80),.05+.2*j0(beta*grid.x),method='BDF',rtol=1e-10,atol=1e-12,
        jac_sparsity=grid.scalar_pattern,events=event)
    assert result.success and result.status==1 and abs(result.t[-1]-exact)<1e-3
    fixed=grid.rate(np.full(161,2.55),1e-9,.015,0.,0.)
    assert np.max(np.abs(fixed))==0
    output=dict(inputs=snapshots,runs=checks,jacobian_checks=jac_errors,analytic_event_s=float(result.t[-1]),exact_event_s=exact,
        sealed_uniform_error=float(np.max(np.abs(fixed))))
    (ROOT/'.work').mkdir(exist_ok=True)

```

```

(ROOT/'.work/smoke.json').write_text(json.dumps(output,ensure_ascii=False,indent=2),encoding='utf-8')
print(json.dumps({k:v for k,v in output.items() if k!='runs'},ensure_ascii=False))
for row in checks: print(json.dumps({k:v for k,v in row.items() if not
k.startswith('sample')},ensure_ascii=False))

if __name__=='__main__':
    parser=argparse.ArgumentParser()
    parser.add_argument('--smoke',action='store_true')
    args=parser.parse_args()
    if args.smoke: smoke()
    else: parser.error('Use --smoke, or run the complete pipeline with run_all.py')

```

B.2 全流程计算与结果导出

```

"""Reproduce numerical tables, convergence, sensitivity and figures."""
from pathlib import Path
import argparse
import copy
import json
import subprocess
import sys
import time
import warnings
from solve import ROOT, SEED, Scenario, integrate, read_inputs
import numpy as np
import pandas as pd
from openpyxl import load_workbook
from export_result2_full import export_run, sample as sample_q3

RESULTS=ROOT/'results'

def emit(value):
    print(json.dumps(value,ensure_ascii=False),flush=True)

def sampling_times(run,step,initial=False):
    return np.unique(np.r_[0. if initial else [],np.arange(step,run.end,step),run.end])

def profile(run,times,radii=None,with_surface=False):
    t,c=run.sample(times,radii)
    if with_surface:
        y=run.values(times);m=run.model.grid.n+1
        t=np.c_[t,y[m-1]];c=np.c_[c,y[2*m-1]]
    return t,c

def compare(coarse,fine,q):
    end=min(coarse.end,fine.end)
    times=np.arange(1,1801) if q==1 else np.arange(1,int(end)+1) if q==2 else
    np.r_[np.arange(60,end,60),end]
    radii=np.arange(20)/10 if q==4 else np.arange(21)/10
    dt=dc=0.
    for start in range(0,len(times),1024):
        block=times[start:start+1024]
        at,ac=sample_q3(coarse,block) if q==2 else profile(coarse,block,radii,q==4)
        bt,bc=sample_q3(fine,block) if q==2 else profile(fine,block,radii,q==4)
        dt=max(dt,float(np.nanmax(abs(at-bt))))

```

```

        dc=max(dc,float(np.nanmax(abs(ac-bc))))
event_diff=None if q in (1,2) else abs(coarse.crossing-fine.crossing)/fine.crossing
return dict(question=q,coarse_n=coarse.model.grid.n,fine_n=fine.model.grid.n,
            temperature_max_difference=dt,moisture_max_difference=dc,
            event_relative_difference=event_diff,sampled_times=len(times),
            passed=dt<.005 and dc<5e-5 and (event_diff is None or event_diff<.001))

def write_book(number,times,arrays,headers):
    template=ROOT/'附件/附件 3'/f'result{number}.xlsx'
    wb=load_workbook(template)
    for ws,values in zip(wb.worksheets,arrays):
        style=copy.copy(ws['B2'].style)
        ws.cell(1,1,'时间\到药材中心的距离')
        for col,label in enumerate(headers,2):
            ws.cell(1,col,label)
        for row,(instant,values_row) in enumerate(zip(times,values),2):
            ws.cell(row,1,int(round(instant)))
            for col,value in enumerate(values_row,2):
                cell=ws.cell(row,col)
                cell.style=copy.copy(style)
                cell.value=None if np.isnan(value) else float(f'{value:.4f}')
                cell.number_format='0.0000'
            ws.freeze_panes='B2'
    target=RESULTS/f'result{number}.xlsx'
    wb.save(target)
    check=load_workbook(target,read_only=True,data_only=True)
    for ws in check.worksheets:
        assert ws.max_row==len(times)+1 and ws.max_column==len(headers)+1
    return dict(path=target.name,rows=len(times),columns=len(headers)+1,bytes=target.stat().st_size)

def save_tables(runs,q2_horizon):
    outputs=[]
    for q in (1,2,3,4):
        run=runs[3] if q==2 else runs[q]
        end=10800 if q==2 and q2_horizon=='3h' else run.end
        times=np.arange(1,int(end)+1) if q in (1,2) else sampling_times(run,60)
        radii=np.arange(20 if q==4 else 21)/10
        headers=radii.tolist()+(['药材表面'] if q==4 else [])
        if q==2:
            outputs.append(export_run(run,RESULTS,rows=int(end)))
        else:
            t,c=profile(run,times,radii,q==4)
            outputs.append(write_book(q,times,[t,c] if q==1 else [c],headers))
        if q==4:
            pd.DataFrame({'time_s':times,'radius_cm':run.model.radius(times)*100}).to_csv(RESULTS/'q4_rad'
            ius.csv',index=False)
        if q==1: paper_times=np.array([100,300,600,900,1200,1500,1800])
        elif q==2: paper_times=np.arange(1800,10801,1800)
        else: paper_times=sampling_times(run,21600)
        rad=np.array([0,.5,1,1.5]) if q==4 else np.arange(5)/2
        pt,pc=profile(run,paper_times,rad,q==4)
        cols=[f'r_{v:g}_cm' for v in rad]+(['surface'] if q==4 else [])
        for tab,values in [(1,pt),(2,pc)] if q==1 else [(3,pt),(4,pc)] if q==2 else [(5 if q==3 else
        6,pc)]:
            frame=pd.DataFrame(values,columns=cols)
            frame.insert(0,'time_s' if q==1 else 'time_h',paper_times if q==1 else paper_times/3600)

```

```

        frame.to_csv(RESULTS/f'table{tab}.csv',index=False,float_format='%.10g')
    return outputs

def save_fields(runs):
    for q,run in runs.items():
        times=sampling_times(run,10 if q==1 else 60,initial=True)
        xi=np.linspace(0,1,101)
        t,c=run.sample(times,xi,normalized=True)
        y=run.values(times);m=run.model.grid.n+1
        cmax=y[m:2*m].max(axis=0);cmean=2*run.model.grid.w@y[m:2*m]
        np.savez_compressed(RESULTS/f'q{q}_fields.npz',time_s=times,x=xi,
                           radius_cm=run.model.radius(times)*100,temperature=t,moisture=c,
                           Cmax=cmax,Cmean=cmean,Csurface=y[2*m-1],Tcenter=y[0],Tsurface=y[m-1],
                           balance_error=cmean+y[-1]-2.55)
        pd.DataFrame(dict(time_s=times,Cmax=cmax,Cmean=cmean,Csurface=y[2*m-1],
                           Tcenter=y[0],Tsurface=y[m-1])).to_csv(RESULTS/f'q{q}_history.csv',index=False)

def compute(q2_horizon='full'):
    started=time.perf_counter();RESULTS.mkdir(exist_ok=True)
    arrays,snapshot=read_inputs()
    pd.DataFrame(arrays[0],columns=['time_s','temperature_C','moisture_kg_kg']).to_csv(RESULTS/'input_env_ironment.csv',index=False)
    pd.DataFrame(arrays[1],columns=['time_s','radius_cm']).to_csv(RESULTS/'input_radius.csv',index=False)
    runs_by_grid={};summaries=[];comparisons=[];last=None
    for n in (160,320,640,1280,2560):
        current={}
        for q in (1,3,4):
            current[q]=integrate(q,n,arrays)
            if q in (3,4) and current[q].crossing is None:
                raise RuntimeError(f'q{q} threshold not reached within prescribed search range')
            row=current[q].summary();summaries.append(row);emit(row)
        if last is not None:
            batch=[compare(last[1] if q==1 else last[4] if q==4 else last[3],
                           current[1] if q==1 else current[4] if q==4 else current[3],q) for q in
                   (1,2,3,4)]
            comparisons.extend(batch);emit({'comparisons':batch})
            if n>=640 and all(r['passed'] for r in batch): break
        last=current
    else: raise RuntimeError('Spatial convergence targets not met up to N2560')
    pd.DataFrame(summaries).to_csv(RESULTS/'grid_runs.csv',index=False)
    pd.DataFrame(comparisons).to_csv(RESULTS/'grid_comparison.csv',index=False)
    final=current;n=final[1].model.grid.n
    time_checks=[]
    for q in (1,3,4):
        tight=integrate(q,n,arrays,rtol=1e-9)
        row=compare(final[q],tight,q)
        row['check']='time_tolerance_1e-8_vs_1e-9'
        time_checks.append(row)
    if q==3:
        second=compare(final[q],tight,2);second['check']=row['check'];time_checks.append(second)
        if not second['passed']:raise RuntimeError('Time tolerance convergence failed q2')
        if not row['passed']:raise RuntimeError(f'Time tolerance convergence failed q{q}')
    pd.DataFrame(time_checks).to_csv(RESULTS/'time_comparison.csv',index=False)
    books=save_tables(final,q2_horizon);save_fields(final)
    scenarios=[('last',Scenario(tail='last')),('mean',Scenario(tail='mean')),
               ('T_minus_0.5',Scenario(temperature_shift=-.5)),('T_plus_0.5',Scenario(temperature_shift=.5)),
               ]

```

```

        ('Ce_minus_0.001', Scenario(moisture_shift=-.001)), ('Ce_plus_0.001', Scenario(moisture_shift
        =.001)),
        ('hm_minus_10pct', Scenario(mass_factor=.9)), ('hm_plus_10pct', Scenario(mass_factor=1.1)))
sensitivity=[]
for q in (3,4):
    baseline=final[q].crossing
    sensitivity.append(dict(question=q,scenario='nominal',crossing_h=baseline/3600,change_pct=0.))
    for name,settings in scenarios:
        run=integrate(q,n,arrays,scenario=settings)
        if run.crossing is None: raise RuntimeError(f'Sensitivity {q}/{name} exceeds data range')
        row=dict(question=q,scenario=name,crossing_h=run.crossing/3600,
                change_pct=100*(run.crossing/baseline-1))
        sensitivity.append(row);emit(row)
counter=integrate(4,n,arrays,horizon=604800,scenario=Scenario(fixed_radius=True))
counter_info=counter.summary();emit({'counterfactual':counter_info})
sensitivity.append(dict(question=4,scenario='fixed_R_appendix4',crossing_h=counter.crossing/3600 if
counter.crossing else None,
                        change_pct=100*(counter.crossing/final[4].crossing-1) if counter.crossing
                        else None))
pd.DataFrame(sensitivity).to_csv(RESULTS/'sensitivity.csv',index=False)
report=dict(seed=SEED,grid_n=n,rtol=1e-8,q2_horizon=q2_horizon,
            runs={str(q):run.summary() for q,run in final.items()},
            output_grid_diagnostics={str(q):run.summary(sample_step_s=1 if q in (1,3) else 60)
                                    for q,run in final.items()},
            counterfactual=counter_info,grid_comparisons=comparisons,time_comparisons=time_checks,
            workbooks=books,compute_runtime_s=time.perf_counter()-started)
(RESULTS/'summary.json').write_text(json.dumps(report,ensure_ascii=False,indent=2),encoding='utf-8')
emit({'compute_complete':report})

def main():
    warnings.filterwarnings('error',category=RuntimeWarning)
    parser=argparse.ArgumentParser()
    parser.add_argument('--compute-only',action='store_true')
    parser.add_argument('--figures-only',action='store_true')
    parser.add_argument('--q2-horizon',choices=['3h','full'],default='full')
    args=parser.parse_args()
    if not args.figures_only:
        if (RESULTS/'result1.xlsx').exists() or (RESULTS/'result2.xlsx').exists():
            raise FileExistsError('Results already exist; use reproduce.py for a fresh isolated run')
        subprocess.run([sys.executable,'-B',str(ROOT/'verify_moving_domain.py'),
                        '--output-dir',str(RESULTS)],check=True)
        compute(args.q2_horizon)
    if not args.compute_only:
        from make_figures import make_figures
        make_figures()
        script=ROOT/'utils/repro_manifest.py'
        command=[sys.executable,str(script),'--project-root',str(ROOT),'--seed',str(SEED),
                '--parameters',json.dumps({'grid_n':json.loads((RESULTS/'summary.json').read_text())['gr
                id_n'],
                'rtol':1e-8,'q2_horizon':args.q2_horizon,'mesh':'1-(1-s)^2'}),
                '--command',f'OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 run_all.py --q2-horizon
                {args.q2_horizon}','--overwrite']
        for path in [ROOT/'A题.pdf',*sorted((ROOT/'附件').rglob('*.xlsx'))]:
            command.extend(['--input',str(path)])
        for package in
            ['numpy','scipy','pandas','openpyxl','matplotlib']:command.extend(['--package',package])

```

```

env=dict(__import__('os').environ)
env['PYTHONPATH']=str(ROOT/'work/deps')+__import__('os').pathsep+env.get('PYTHONPATH','')
subprocess.run(command,check=True,env=env)

if __name__=='__main__':main()

```

B.3 真实结果可视化

```

"""Generate evidence-led figures from completed, unrounded result files."""
from pathlib import Path
import json
import logging
import os
import sys
from solve import ROOT, properties
os.environ.setdefault('MPLCONFIGDIR', str(ROOT/'work/mplconfig'))
os.environ.setdefault('SOURCE_DATE_EPOCH', '1788998400')
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.colors import LogNorm
from matplotlib.ticker import FuncFormatter
from utils.setup_style import setup_style, CJK_FONT_PRIORITY
from utils.plot_style import export_figure as audited_export
from utils.export_figure import export_figure as pdf_export

OUT=ROOT/'figures'
DATA=ROOT/'results'
BLUE='#0072B2'; ORANGE='#D55E00'; GREEN='#009E73'; GRAY='#666666'
COLORS=[BLUE,ORANGE,GREEN]
LINES=['-','-','-']
WIDTH=6.3

class GlyphFailureHandler(logging.Handler):
    def emit(self,record):
        message=record.getMessage()
        if 'does not have a glyph' in message or 'missing from font' in message:
            raise RuntimeError(message)

def configure():
    logging.getLogger('matplotlib').addHandler(GlyphFailureHandler())
    sources=[ROOT/'assets/wqy-zenhei.ttc',
             ROOT/'work/tex_runtime/usr/share/fonts/truetype/wqy/wqy-zenhei.ttc',
             Path('/usr/share/fonts/truetype/wqy/wqy-zenhei.ttc')]
    source=next((p for p in sources if p.exists()),None)
    if source is None:
        raise RuntimeError('Install fonts-wqy-zenhei or supply assets/wqy-zenhei.ttc')
    font_manager.fontManager.addfont(str(source))
    CJK_FONT_PRIORITY.insert(0,'WenQuanYi Zen Hei')
    setup_style(journal='general',lang='zh',use_sciplots=False)
    plt.rcParams.update({'font.size':8.5,'axes.labelsize':9,'axes.titlesize':9,
                        'legend.fontsize':8,'xtick.labelsize':8,'ytick.labelsize':8,

```

```

        'axes.titlelocation':'left','axes.spines.top':False,
        'axes.spines.right':False,'lines.linewidth':1.4,
        'figure.facecolor':'white','axes.facecolor':'white',
        'legend.frameon':False,'savefig.dpi':300,'svg.hashsalt':'20260910'})
OUT.mkdir(exist_ok=True)

def panels(count=1,height=2.9):
    return plt.subplots(1,count,figsize=(WIDTH,height),layout='constrained',squeeze=False)[0:2]

def save(fig,stem):
    for ax in fig.axes:
        ax.minorticks_off()
        if ax.get_yscale()=='log' and not hasattr(ax,'_colorbar'):
            ax.yaxis.set_major_formatter(FuncFormatter(lambda x,pos:f'{x:.0e}'))
        if hasattr(ax,'_colorbar') and ax._colorbar.solids is not None:
            ax._colorbar.solids.set_rasterized(False)
    audited_export(fig,OUT/stem,dpi=300,strict_layout=True,strict_design=True)
    pdf_export(fig,str(OUT/stem),formats=['pdf'],dpi=300,
                size_inches=tuple(fig.get_size_inches()),grayscale_preview=False,tight=False)
    plt.close(fig)

def series(ax,x,ys,labels):
    for y,label,color,ls in zip(ys,labels,COLORS,LINES):
        ax.plot(x,y,label=label,color=color,ls=ls)
    ax.legend()
    ax.margins(x=0.01)

def make_figures():
    configure()
    env=pd.read_csv(DATA/'input_environment.csv')
    rad=pd.read_csv(DATA/'input_radius.csv')
    summary=json.loads((DATA/'summary.json').read_text())
    q1=np.load(DATA/'q1_fields.npz');q3=np.load(DATA/'q3_fields.npz');q4=np.load(DATA/'q4_fields.npz')

    fig,axes=panels(2);a,b=axes[0]
    a.plot(env.time_s/3600,env.temperature_C,color=BLUE)
    b.plot(env.time_s/3600,env.moisture_kg_kg,color=ORANGE)
    for ax in (a,b):ax.set_xlabel('时间 / h');ax.set_xlim(0,4)
    a.set_ylabel('环境温度 / ℃');b.set_ylabel('有效平衡含水率 / (kg/kg)')
    a.set_title('(a) 温度边界');b.set_title('(b) 水分边界')
    save(fig,'raw_q1_environment')

    tail=env[env.time_s>=10800]
    fig,axes=panels(2);a,b=axes[0]
    a.hist(tail.temperature_C,bins=8,color=BLUE,alpha=.75,edgecolor='white')
    b.hist(tail.moisture_kg_kg,bins=8,color=ORANGE,alpha=.75,edgecolor='white')
    a.axvline(50,color='black',ls='--',lw=1);b.axvline(.05,color='black',ls='--',lw=1)
    a.set_xlabel('环境温度 / ℃');b.set_xlabel('有效平衡含水率 / (kg/kg)')
    for ax in (a,b):ax.set_ylabel('频数');ax.set_title('末小时: 61 个记录')
    b.ticklabel_format(axis='x',style='plain',useOffset=False)
    b.set_xticks([.0498,.05,.0502])
    save(fig,'raw_q2_tail')

    fig,axes=panels(height=2.7);a=axes[0,0];c=np.linspace(.05,2.55,501)
    for t,ls,col in ((28,'--',BLUE),(50,'-',ORANGE)):
        a.semilogy(c,properties(3,np.full_like(c,t),c)[2],color=col,ls=ls,label=f'{t}℃')

```

```

a.set_xlabel('干基含水率 C / (kg/kg)');a.set_ylabel('扩散系数 D / (m²/s)')
a.legend();a.set_title('题设物性函数 (非实测) ')
save(fig,'raw_q3_diffusivity')

fig,axes=panels(height=2.7);a=axes[0,0]
a.plot(rad.time_s/3600,rad.radius_cm,color=BLUE,lw=1)
a.scatter(rad.time_s/3600,rad.radius_cm,s=7,color=BLUE)
a.set_xlabel('时间 / h');a.set_ylabel('药材半径 / cm');a.set_xlim(0,72)
a.set_title('题给半径: 145 个记录')
save(fig,'raw_q4_radius')

fig,axes=panels(height=2.7);a=axes[0,0]
times=q1['time_s']
series(a,times/60,[q1['Tcenter'],q1['Tsurface']],np.interp(times,env.time_s,env.temperature_C)),
    ['中心','表面','环境'])
a.set_xlabel('时间 / min');a.set_ylabel('温度 / °C')
save(fig,'process_q1_heating')

fig,axes=panels(2);a,b=axes[0]
for t,color,ls in zip([100,600,1800],COLORS,LINES):
    i=np.flatnonzero(times==t)[0]
    a.plot(q1['x']*2,q1['temperature'][i],color=color,ls=ls,label=f'{t} s')
    b.plot(q1['x']*2,q1['moisture'][i],color=color,ls=ls,label=f'{t} s')
a.set_ylabel('温度 / °C');b.set_ylabel('干基含水率 / (kg/kg)')
for ax in (a,b):ax.set_xlabel('半径位置 / cm');ax.legend()
a.set_title('(a) 温度');b.set_title('(b) 水分')
save(fig,'result_q1_profiles')

take=q3['time_s']<=10800;tt=q3['time_s'][take]/3600;xx=q3['x']*2
temps=q3['temperature'][take];moist=q3['moisture'][take]
diff=properties(3,temps,moist)[2]
fig,axes=panels(height=2.9);a=axes[0,0]
mesh=a.pcolormesh(tt,xx,diff.T,norm=LogNorm(),cmap='viridis',shading='auto')
cb=fig.colorbar(mesh,ax=a,ticks=np.geomspace(diff.min(),diff.max(),4),
    format=FuncFormatter(lambda x,pos:f'{x:.1e}'))
cb.minorticks_off();cb.set_label('D / (m²/s), 对数色标')
a.set_xlabel('时间 / h');a.set_ylabel('半径位置 / cm')
save(fig,'process_q2_diffusion')

fig,axes=panels(2,height=3.1);a,b=axes[0]
for ax,field,label in [(a,temps,'温度 / °C'),(b,moist,'C / (kg/kg)')]:
    mesh=ax.pcolormesh(tt,xx,field.T,cmap='viridis',shading='auto')
    cb=fig.colorbar(mesh,ax=ax,pad=.02);cb.set_label(label)
    ax.set_xlabel('时间 / h');ax.set_ylabel('半径位置 / cm')
a.set_title('(a) 温度场');b.set_title('(b) 含水率场')
save(fig,'result_q2_fields')

fig,axes=panels(height=2.9);a=axes[0,0]
series(a,q3['time_s']/3600,[q3['Cmax'],q3['Cmean'],q3['Csurface']],['全域最大','径向体积权均值','表面'])
a.axhline(.15,color='black',ls=':',lw=1.1)
a.set_xlabel('时间 / h');a.set_ylabel('干基含水率 / (kg/kg)')
save(fig,'process_q3_threshold')

fig,axes=panels(height=2.7);a=axes[0,0]
a.plot(xx,q3['moisture'][-1],color=BLUE,label='严格终点的含水率')
a.axhline(.15,color='black',ls='--',label='阈值 0.15')

```

```

a.set_xlabel('半径位置 / cm');a.set_ylabel('干基含水率 / (kg/kg)')
a.legend();a.set_ylim(0,.17)
save(fig,'result_q3_terminal')

fig,axes=panels(height=3.1);a=axes[0,0]
ts=q4['time_s']/3600;rs=q4['radius_cm'];rr=np.linspace(0,2,161)
physical=np.full((len(ts),len(rr)),np.nan)
for i,radius in enumerate(rs):
    inside=rr<=radius
    physical[i,inside]=np.interp(rr[inside]/radius,q4['x'],q4['moisture'][i])
mesh=a.contourf(ts,rr,np.ma.masked_invalid(physical.T),cmap='viridis',
    levels=np.linspace(0,2.55,22),corner_mask=False)
fig.colorbar(mesh,ax=a).set_label('C / (kg/kg)')
a.plot(ts,rs,color='black',lw=1,label='实际表面')
a.set_xlabel('时间 / h');a.set_ylabel('物理半径位置 / cm');a.legend(loc='upper right')
save(fig,'process_q4_moving')

fig,axes=panels(height=2.7);a=axes[0,0]
labels=['问题 3: 固定半径','问题 4 物性: 固定半径','问题 4: 给定收缩']
hours=[summary['runs']['3']['crossing_h'],summary['counterfactual']['crossing_h'],
    summary['runs']['4']['crossing_h']]
for i,(value,color) in enumerate(zip(hours,COLORS)):
    a.scatter(value,i,color=color,s=32,zorder=3)
    a.annotate(f'{value:.3f} h',(value,i),xytext=(6,5),textcoords='offset points',fontsize=8)
a.set_yticks(range(3),labels);a.set_xlabel('全域达到阈值的事件时间 / h')
a.set_xlim(0,max(hours)*1.18);a.set_ylim(2.5,-.5);a.grid(axis='x',alpha=.2)
save(fig,'result_q4_counterfactual')

grid=pd.read_csv(DATA/'grid_comparison.csv')
fig,axes=panels(2);a,b=axes[0]
for q,color,ls in zip([1,2,3,4],[BLUE,ORANGE,GREEN,GRAY],[-','--','-','.:']):
    frame=grid[grid.question==q]
    a.loglog(frame.fine_n,frame.moisture_max_difference,color=color,ls=ls,marker='o',label=f'问题 {q}')
    b.loglog(frame.fine_n,frame.temperature_max_difference,color=color,ls=ls,marker='o',label=f'问题 {q}')
a.axhline(5e-5,color='black',lw=.8);b.axhline(.005,color='black',lw=.8)
for ax in (a,b):
    ax.set_xlabel('细网格区间数 N');ax.legend(ncol=2)
    ticks=sorted(grid.fine_n.unique());ax.set_xticks(ticks,[str(t) for t in ticks])
a.set_ylabel('含水率最大差 / (kg/kg)');b.set_ylabel('温度最大差 / °C')
save(fig,'process_q3_convergence')

sens=pd.read_csv(DATA/'sensitivity.csv')
names=['last','mean','T_minus_0.5','T_plus_0.5','Ce_minus_0.001','Ce_plus_0.001','hm_minus_10pct','hm_
_plus_10pct']
labels=['末值延拓','末小时均值延拓','尾段温度 -0.5°C','尾段温度 +0.5°C','尾段平衡值 -0.001','尾段平衡值
+0.001','传质系数 -10%','传质系数 +10%']
fig,axes=panels(height=3.2);a=axes[0,0]
for q,col,offset,marker in [(3,BLUE,-.12,'o'),(4,ORANGE,.12,'s')]:
    frame=sens[sens.question==q].set_index('scenario')
    vals=frame.loc[names,'change_pct'].values
    a.scatter(vals,np.arange(8)+offset,color=col,marker=marker,s=22,label=f'问题 {q}')
a.set_yticks(range(8),labels);a.invert_yaxis();a.axvline(0,color='black',lw=.7)
a.set_xlabel('事件时间相对名义情景的变化 / %');a.legend(loc='lower right')
save(fig,'result_q4_sensitivity')
print(json.dumps({'figures':len(list(OUT.glob('*.svg'))),'format':['svg','png','pdf'],'dpi':300}))

```

```
if __name__=='__main__':make_figures()
```

B.4 移动域解析基准与几何因子对照

```
"""Moving Neumann/Bessel benchmark of the unchanged production radial operator.
```

```
No output files unless --output-dir is supplied. --mutant-only MUST exit 1.
```

```
Run from the synchronized project: python3 -B verify_moving_domain.py
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import csv
```

```
import hashlib
```

```
import json
```

```
from pathlib import Path
```

```
import sys
```

```
import time
```

```
import warnings
```

```
HERE = Path(__file__).resolve().parent
```

```
PROJECT = HERE
```

```
sys.path.insert(0, str(PROJECT))
```

```
from solve import Grid, SEED, integrate, read_inputs
```

```
import numpy as np
```

```
from scipy.integrate import solve_ivp
```

```
from scipy.linalg import eigh_tridiagonal
```

```
from scipy.special import j0, j1, jn_zeros
```

```
R0, TF, D, C0, AMP = 0.02, 3600.0, 1e-8, 1.0, 0.3
```

```
BETA = 1.0 / (2 * TF)
```

```
LAM = float(jn_zeros(1, 1)[0])
```

```
TIMES = np.linspace(0, TF, 129)
```

```
SPACE_LIMIT, TIME_LIMIT, MASS_LIMIT = 1e-5, 5e-6, 1e-10
```

```
def radius(t):
```

```
    return R0 * (1 - BETA * np.asarray(t))
```

```
def clock_integral(t):
```

```
    t = np.asarray(t)
```

```
    return t / (R0**2 * (1 - BETA * t))
```

```
def exact(x, times=TIMES):
```

```
    return C0 + AMP * j0(LAM * np.asarray(x)[: , None]) * np.exp(
        -LAM**2 * D * clock_integral(times)[None, :])
```

```
)
```

```

def semidiscrete_reference(grid, initial, times=TIMES):
    """Matrix exponential via the weighted-symmetric tridiagonal operator.

    Used on a fixed UNIFORM spatial grid to isolate temporal error. The actual
    PDE/Bessel comparison is reported independently; no fitted reference data.
    """
    lap = grid.jac_block(initial, 1.0, 0.0, 1.0, 0.0, same=True)
    sw = np.sqrt(grid.w)
    upper = lap.diagonal(1) * sw[:-1] / sw[1:]
    lower = lap.diagonal(-1) * sw[1:] / sw[:-1]
    assert np.allclose(upper, lower, rtol=1e-13, atol=1e-10)
    vals, vecs = eigh_tridiagonal(lap.diagonal(), upper)
    assert abs(vals[-1]) < 1e-7 and vals[-2] < 0
    # The zero eigenvalue is known exactly from the conservative Neumann matrix.
    vals[-1] = 0.0
    coeff = vecs.T @ (sw * initial)
    answer = (vecs @ (coeff[:, None] * np.exp(
        vals[:, None] * D * clock_integral(times)[None, :])
    ))) / sw[:, None]
    assert np.max(abs(answer[:, 0] - initial)) < 1e-11
    return answer

def solve_case(n, *, graded=True, rtol=1e-11, max_step=TF/256,
               mutant=False, temporal_reference=False):
    grid = Grid(n, graded=graded)
    initial = exact(grid.x, np.array([0.0]))[:, 0]
    # jac_block and rate both come from solve.py, never copied/replaced locally.
    lap = grid.jac_block(initial, D, 0.0, 1.0, 0.0, same=True)

    def r_used(t):
        return R0 if mutant else float(radius(t))

    def rhs(t, c):
        return grid.rate(c, D, r_used(t), 0.0, 0.0)

    result = solve_ivp(rhs, (0, TF), initial, method="BDF", rtol=rtol,
                       atol=rtol*0.01, max_step=max_step,
                       jac=lambda t, c: lap/r_used(t)**2, dense_output=True)
    if not result.success:
        raise RuntimeError(result.message)
    values = result.sol(TIMES)
    reference = exact(grid.x)
    mass0 = float(2*grid.w @ initial)
    # Check every accepted step as well as all shared comparison instants.
    mass_drift = max(float(np.max(abs(2*grid.w @ values-mass0))),
                     float(np.max(abs(2*grid.w @ result.y-mass0))))
    continuum_error = float(np.max(abs(values-reference)))
    temporal_error = None
    if temporal_reference:
        temporal_error = float(np.max(abs(values-semidiscrete_reference(grid, initial))))
    row = dict(n=n, graded=graded, rtol=rtol, atol=rtol*0.01,
               max_step_s=max_step, accepted_steps=len(result.t)-1,
               actual_max_step_s=float(np.diff(result.t).max()),

```

```

        continuum_max_error=continuum_error, temporal_max_error=temporal_error,
        mass_drift_max=mass_drift, initial_mean_quadrature_error=mass0-C0,
        numerical_center_final=float(values[0,-1]),
        exact_center_final=float(reference[0,-1]),
        mutant=mutant, nfev=result.nfev)

    return row

def regression_checks():
    arrays, snapshots = read_inputs()
    rows = []
    for q in (1, 3, 4):
        run = integrate(q, 80, arrays, horizon=600, stop_at_threshold=False)
        times = np.unique(np.concatenate([np.arange(0, 601)]
                                          + [s.t for s in run.segments]))

        values = run.values(times)
        m = run.model.grid.n+1
        residual = float(np.max(abs(2*run.model.grid.w@values[m:2*m]
                                   +values[-1]-2.55)))

        closed = run.model.grid.rate(np.full(m, 2.55), 1e-9, 0.015, 0, 0)
        rows.append(dict(question=q, mass_balance_max=residual,
                        closed_uniform_rate_max=float(np.max(abs(closed))),
                        radius_start_m=float(run.model.radius(0)),
                        radius_end_m=float(run.model.radius(600)),
                        passed=bool(residual < 1e-7 and np.max(abs(closed)) == 0)))

    return rows, snapshots

def run_suite():
    start = time.perf_counter()
    space = [solve_case(n) for n in (40, 80, 160, 320, 640)]
    temporal = [solve_case(160, graded=False, rtol=tol, max_step=step,
                          temporal_reference=True)
               for tol, step in ((1e-3, TF/8), (1e-5, TF/16), (1e-7, TF/32))]
    mutant = solve_case(640, mutant=True)
    regressions, snapshots = regression_checks()
    se = [r["continuum_max_error"] for r in space]
    te = [r["temporal_max_error"] for r in temporal]
    checks = dict(
        radius_changes=radius(TF) != radius(0),
        nonuniform_initial=np.ptp(exact(np.linspace(0,1,101),np.array([0]))) > 0.1,
        neumann_eigenvalue=abs(j1(LAM)) < 1e-12,
        correct_operator=se[-1] <= SPACE_LIMIT,
        space_errors_decrease=all(b < a for a,b in zip(se,se[1:])),
        time_errors_decrease=all(b < a for a,b in zip(te,te[1:])),
        time_grid_refines=all(b["accepted_steps"] > a["accepted_steps"]
                              for a,b in zip(temporal,temporal[1:])),
        finest_temporal_error=te[-1] <= TIME_LIMIT,
        mutant_rejected=mutant["continuum_max_error"] >= 100*SPACE_LIMIT,
        closed_mass_preserved=all(r["mass_drift_max"] <= MASS_LIMIT
                                  for r in space+temporal+[mutant]),
        original_mass_regression=all(r["passed"] for r in regressions),
    )
    checks = {k:bool(v) for k,v in checks.items()}
    return dict(status="PASS" if all(checks.values()) else "FAIL", checks=checks,
               parameters=dict(R0_m=R0, final_radius_m=float(radius(TF)), tf_s=TF,

```

```

        D_m2_s=D, C0=C0, amplitude=AMP, beta_per_s=BETA,
        eigenvalue=LAM, comparison_times_s=TIMES.tolist(),
        spatial_error_limit=SPACE_LIMIT, time_error_limit=TIME_LIMIT,
        closed_mass_limit=MASS_LIMIT),
    spatial=space, temporal=temporal, mutant=mutant,
    mass_regressions=regressions, inputs=snapshots, seed=SEED,
    production_solver_sha256=hashlib.sha256((PROJECT/"solve.py").read_bytes()).hexdigest(),
    benchmark_sha256=hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),
    runtime_s=time.perf_counter()-start)

def main():
    warnings.filterwarnings("error", category=RuntimeWarning)
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output-dir", type=Path)
    parser.add_argument("--mutant-only", action="store_true")
    args = parser.parse_args()
    if args.mutant_only:
        row = solve_case(640, mutant=True)
        print(json.dumps(row, ensure_ascii=False, indent=2))
        return 1 if row["continuum_max_error"] > SPACE_LIMIT else 0
    report = run_suite()
    if args.output_dir:
        out = args.output_dir.resolve()
        if PROJECT not in out.parents:
            raise ValueError("Outputs must stay below A 题")
        out.mkdir(parents=True, exist_ok=True)
        (out/"moving_domain_benchmark.json").write_text(
            json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
        with (out/"moving_domain_convergence.csv").open("w", newline="", encoding="utf-8") as stream:
            rows = [dict(kind=k, **r) for k, group in
                    (("space",report["spatial"]),("time",report["temporal"]),
                     ("mutant",[report["mutant"]])) for r in group]
            writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
            writer.writeheader(); writer.writerows(rows)
        print(json.dumps(report, ensure_ascii=False, indent=2))
        return 0 if report["status"] == "PASS" else 1

if __name__ == "__main__":
    sys.exit(main())

```

B.5 全过程逐秒工作簿导出与核对

"""Template-preserving, bounded-memory export of q2 from the q3 dense solution.

Imported by run_all.py; the complete isolated entry point is reproduce.py.

--verify-only audits an existing workbook without writing it.

"""

from __future__ import annotations

import argparse

from copy import copy

import csv

import hashlib

from io import BytesIO

```

import json
from pathlib import Path
import re
import sys
import time
import warnings
from xml.etree import ElementTree as ET
from zipfile import ZipFile, ZIP_DEFLATED

HERE = Path(__file__).resolve().parent
PROJECT = HERE
sys.path.insert(0, str(PROJECT))
from solve import SEED, integrate, read_inputs
from utils.repro_manifest import build_manifest
import numpy as np
from scipy.interpolate import PchipInterpolator
from openpyxl import load_workbook
from openpyxl.utils import get_column_letter

NS = "{http://schemas.openxmlformats.org/spreadsheetml/2006/main}"
COLS = [get_column_letter(i) for i in range(1,23)]
RADII = np.arange(21)/10

def sha(path):
    digest = hashlib.sha256()
    with Path(path).open("rb") as stream:
        for block in iter(lambda: stream.read(1024*1024), b''):
            digest.update(block)
    return digest.hexdigest()

def sample(run, times):
    """Same PCHIP as Run.sample, vectorized over a bounded time block."""
    times = np.asarray(times)
    assert run.model.q == 3 and np.all(run.model.radius(times) == .02)
    state = run.values(times)
    m = run.model.grid.n+1
    coords = RADII/100/.02
    t = PchipInterpolator(run.model.grid.x, state[:m], axis=0)(coords).T
    c = PchipInterpolator(run.model.grid.x, state[m:2*m], axis=0)(coords).T
    assert np.isfinite(t).all() and np.isfinite(c).all()
    return t,c

def template_skeleton():
    """Load and edit the actual template; preserve all non-data ZIP members.

    Only worksheet row data are streamed into this openpyxl-produced skeleton.
    No sheet/name/style/theme relationships are reconstructed by hand.
    """
    wb = load_workbook(PROJECT/" 附件/附件 3/result2.xlsx")
    assert wb.sheetnames == [" 温度", " 水分浓度"]
    assert not wb.defined_names and not wb._external_links
    styles=[]
    for ws in wb:

```

```

assert not ws.merged_cells and not ws._images and not ws._charts
assert not ws.tables and not ws.data_validations.dataValidation
assert not ws.conditional_formatting and not ws.auto_filter.ref
assert all(c.data_type != "f" for row in ws for c in row)
header_style=copy(ws["B1"]._style)
value_style=copy(ws["B2"]._style)
time_style=copy(ws["A2"]._style)
ws.delete_rows(2,ws.max_row-1) # Only placeholders in the in-memory copy.
for col,r in enumerate(RADII,2):
    cell=ws.cell(1,col,float(r));cell._style=copy(header_style)
ws.cell(2,1,1)._style=time_style
for col in range(2,23):
    cell=ws.cell(2,col,0.0);cell._style=copy(value_style)
    cell.number_format="0.0000"
ws.freeze_panes="B2"
styles.append((ws["A2"].style_id,ws["B2"].style_id))
buffer=BytesIO();wb.save(buffer)
return buffer.getvalue(),styles

def write_book(run,path,rows,chunk):
    if path.exists():
        raise FileExistsError(f"Refusing to overwrite {path}")
    skeleton,styles=template_skeleton()
    with ZipFile(BytesIO(skeleton)) as source, ZipFile(path,"w",compression=ZIP_DEFLATED,
                                                    compresslevel=9) as target:
        for item in source.infolist():
            match=re.fullmatch(r"xl/worksheets/sheet([12])\.xml",item.filename)
            if not match:
                target.writestr(item,source.read(item.filename))
                continue
            index=int(match[1])-1
            original=source.read(item.filename).decode("utf-8")
            prefix,rest=original.split("<sheetData>",1)
            data,suffix=rest.split("</sheetData>",1)
            header=data[:data.index("</row>")+len("</row>")]
            prefix=re.sub(r'<dimension ref="[~]+"\s',
                        f'<dimension ref="A1:V{rows+1}"',prefix)
            style_time,style_value=styles[index]
            with target.open(item.filename,"w",force_zip64=True) as stream:
                stream.write((prefix+"<sheetData>"+header).encode("utf-8"))
                for start in range(1,rows+1,chunk):
                    times=np.arange(start,min(rows+1,start+chunk))
                    values=sample(run,times)[index]
                    # Independent scalar production sampler checks vectorization.
                    scalar=run.sample([times[0],times[-1]],RADII)[index]
                    assert np.max(abs(values[[0,-1]]-scalar)) < 1e-12
                    lines=[]
                    for instant,row in zip(times,values):
                        number=int(instant)+1
                        cells=[f'<c r="A{number}" s="{style_time}" t="n"><v>{int(instant)}</v></c>']
                        cells.extend(f'<c r="{col}{number}" s="{style_value}"
                                    t="n"><v>{float(v):.4f}</v></c>'
                                    for col,v in zip(COLS[1:],row))
                        lines.append(f'<row r="{number}">'+"".join(cells)+"</row>")
                    stream.write("".join(lines).encode("utf-8"))

```

```

        if start == 1 or (start-1)//chunk % 16 == 0:
            print(json.dumps({"sheet":index+1,"written_through_s":int(times[-1])}),flush=True)
        stream.write(("</sheetData>" + suffix).encode("utf-8"))

def audit_book(path,rows,expected=None,old_prefix=False):
    """Full XML scan: all cells, row/time order, ranges, plus numerical samples."""
    book=load_workbook(path,read_only=True,data_only=True)
    assert book.sheetnames == [" 温度"," 水分浓度"]
    for ws in book:
        assert ws.max_row == rows+1 and ws.max_column == 22
        first=next(ws.iter_rows(min_row=1,max_row=1,values_only=True))
        assert np.allclose(first[1:],RADII)
    book.close()
    prefixbook=load_workbook(PROJECT/"results/result2.xlsx",read_only=True,data_only=True) if old_prefix
    else None
    output=[]
    with ZipFile(path) as archive:
        for index in (0,1):
            olditer=iter(prefixbook.worksheets[index].iter_rows(min_row=2,values_only=True)) if
            prefixbook else None
            count=0;minimum=float("inf");maximum=-float("inf");sample_count=0;prefix_count=0
            with archive.open(f"xl/worksheets/sheet{index+1}.xml") as stream:
                context=ET.iterparse(stream,events=("start","end"))
                _,root=next(context)
                for event,row in context:
                    if event != "end" or row.tag != NS+"row":
                        continue
                    number=int(row.attrib["r"])
                    if number == 1:
                        row.clear();root.clear();continue
                    count+=1
                    assert number == count+1
                    cells=list(row)
                    assert len(cells)==22
                    assert all(c.attrib["r"]==f"{{col}}-{{number}}" and c.attrib.get("t","n")=="n"
                               for c,col in zip(cells,COLS))
                    vals=[float(c.find(NS+"v").text) for c in cells]
                    assert vals[0] == count
                    assert np.isfinite(vals).all()
                    low,high=min(vals[1:]),max(vals[1:])
                    assert (27.99 <= low <= high <= 50.6) if index==0 else (0 < low <= high <= 2.5501)
                    minimum=min(minimum,low);maximum=max(maximum,high)
                    if expected is not None and count in expected:
                        assert np.array_equal(np.asarray(vals[1:]),np.asarray(expected[count][index]))
                        sample_count+=1
                    if olditer is not None and count <= min(rows,10800):
                        assert tuple(vals)==tuple(next(olditer))
                        prefix_count+=1
                    row.clear();root.clear()
            assert count==rows
            assert expected is None or sample_count==len(expected)
            output.append(dict(sheet=[" 温度"," 水分浓度"][index],data_rows=count,
                               columns=22,minimum=minimum,maximum=maximum,
                               sampled_rows_verified=sample_count,
                               old_3h_rows_identical=prefix_count))

```

```

    if prefixbook:prefixbook.close()
    return output

def export_run(run, out, *, rows=None, chunk=4096):
    """Export the supplied, unrounded q3 solution; no dependency on old outputs."""
    start=time.perf_counter(); out=Path(out).resolve()
    if PROJECT not in out.parents or chunk < 1:
        raise ValueError('Output must be below project root; positive chunk required')
    name='result2.xlsx'
    if rows is None:
        rows=int(run.end)
    if not 1 <= rows <= int(run.end):
        raise ValueError('Invalid export horizon')
    stamps=np.unique(np.r_[np.linspace(1,rows,101).astype(int),
                           [t for t in (1,60,600,1800,10800,14400,14401,rows) if t<=rows]])
    st,sc=sample(run,stamps)
    expected={int(t):[[float(f"{v:.4f}") for v in tt],[float(f"{v:.4f}") for v in cc]]
              for t,tt,cc in zip(stamps,st,sc)}
    out.mkdir(parents=True,exist_ok=True)
    write_book(run,out/name,rows,chunk)
    audit=audit_book(out/name,rows,expected)
    report=dict(status="PASS",mode="full" if rows==int(run.end) else "prefix",rows=rows,
               first_time_s=1,last_time_s=rows,radial_positions_cm=RADII.tolist(),
               production_summary=run.summary(sample_step_s=1),samples=expected,
               audit=audit,output_bytes=(out/name).stat().st_size,output_sha256=sha(out/name),
               exporter_sha256=sha(Path(__file__)),runtime_s=time.perf_counter()-start,
               diagnostic_note='Statistics include t=0 and all solver nodes; Excel starts at 1 s and
                               uses 21 output radii.')
    (out/"result2_export.json").write_text(
        json.dumps(report,ensure_ascii=False,indent=2),encoding="utf-8")
    return dict(path=name,rows=rows,columns=22,bytes=report['output_bytes'])

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--verify-only',action='store_true',required=True)
    parser.add_argument('--output-dir',type=Path,default=HERE/'results')
    args=parser.parse_args()
    report=json.loads((args.output_dir/'result2_export.json').read_text())
    assert sha(args.output_dir/'result2.xlsx')==report['output_sha256']
    audit=audit_book(args.output_dir/'result2.xlsx',report['rows'],
                     expected={int(k):v for k,v in report['samples'].items()})
    print(json.dumps(dict(status='PASS',audit=audit,ensure_ascii=False,indent=2)))
    return 0

if __name__=="__main__":
    sys.exit(main())

```

B.6 完整计算入口

"""Reproduce all four questions in a fresh local directory; never overwrite.

Run from this project: `python3 -B reproduce.py`

Use `--smoke` only for the 600-second integration/export regression tests.

```

"""
from pathlib import Path
import argparse
import hashlib
import json
import os
import shutil
import subprocess
import sys
import tempfile
import time

ROOT=Path(__file__).resolve().parent
SOURCES=('solve.py', 'run_all.py', 'make_figures.py', 'verify_moving_domain.py',
        'export_result2_full.py', 'test_sync.py', 'reproduce.py', 'requirements.txt',
        'A 题.pdf', '题目分析报告.md', '术语表格.md')

def main():
    parser=argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--smoke', action='store_true')
    parser.add_argument('--compute-only', action='store_true')
    args=parser.parse_args()
    start=time.perf_counter()
    out=Path(tempfile.mkdtemp(prefix='reproduced_', dir=ROOT))
    for name in SOURCES:
        shutil.copy2(ROOT/name, out/name)
    for name in ('附件', 'utils', 'assets'):
        shutil.copytree(ROOT/name, out/name, ignore=shutil.ignore_patterns('__pycache__', '*.pyc'))
    env=dict(os.environ, PYTHONDONTWRITEBYTECODE='1', OPENBLAS_NUM_THREADS='1', OMP_NUM_THREADS='1')
    if env.get('PYTHONPATH'):
        env['PYTHONPATH']=os.pathsep.join(str(Path(p or '.').resolve())
                                         for p in env['PYTHONPATH'].split(os.pathsep))
    print(f'Reproduction directory: {out}', flush=True)
    command=[sys.executable, '-B', 'test_sync.py'] if args.smoke else
    [sys.executable, '-B', 'run_all.py', '--q2-horizon', 'full']
    if args.compute_only and not args.smoke:
        command.append('--compute-only')
    subprocess.run(command, cwd=out, env=env, check=True)
    from utils.repro_manifest import build_manifest
    inputs=([ROOT/n for n in SOURCES]+sorted((ROOT/'附件').rglob('*xlsx'))
            +sorted((ROOT/'utils').glob('*py'))+[p for p in sorted((ROOT/'assets').rglob('*')) if
            p.is_file()])
    parameters=(dict(n=80, rtol=1e-8, horizon_s=600, smoke=True)
                if args.smoke else dict(n=json.loads((out/'results/summary.json').read_text())['grid_n'],
                rtol=1e-8, q2_horizon='full', smoke=False, figures=not
                args.compute_only))
    suffix=' --smoke' if args.smoke else ' --compute-only' if args.compute_only else ''
    manifest=build_manifest(inputs, 20260910, parameters,
                           'OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python3 -B reproduce.py'+suffix,
                           ['numpy', 'scipy', 'pandas', 'openpyxl', 'matplotlib'])
    manifest['runtime_s']=time.perf_counter()-start
    manifest['outputs']=[dict(path=str(p.relative_to(out)), bytes=p.stat().st_size,
                             sha256=hashlib.sha256(p.read_bytes()).hexdigest())
                        for folder in ('results', 'figures') if (out/folder).exists()
                        for p in sorted((out/folder).rglob('*'))]

```

```

        if p.is_file() and p.name != '复现清单.json']
    (out/'results').mkdir(exist_ok=True)
    (out/'results/复现清单.json').write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding='utf-8')
    print(json.dumps(dict(status='PASS',output_directory=str(out),runtime_s=manifest['runtime_s']),ensure_
    _ascii=False),flush=True)

if __name__=='__main__':
    main()

```

B.7 采样与导出检查

```

"""Small real-input regression of diagnostic scope and streamed output."""
from pathlib import Path
import json
import tempfile
import numpy as np
from solve import integrate, read_inputs
from export_result2_full import export_run, audit_book

def main():
    arrays,_=read_inputs()
    rows=[]
    for q in (1,3,4):
        run=integrate(q,80,arrays,horizon=600,stop_at_threshold=False)
        sparse=run.summary()
        dense=run.summary(sample_step_s=1,chunk_size=37)
        assert sparse['diagnostic_sampling']['time_count']==250
        assert dense['diagnostic_sampling']['time_count']==601
        assert not dense['diagnostic_sampling']['continuous_extrema_claimed']
        assert not {'T_max','T_min','C_max','C_min','moisture_balance_max','max_above_center'} &
        dense.keys()
        y=run.values(np.arange(601));m=81
        expected={'T_max_sampled':y[:m].max(),'T_min_sampled':y[:m].min(),
                  'C_max_sampled':y[m:2*m].max(),'C_min_sampled':y[m:2*m].min(),
                  'moisture_balance_max_sampled':np.max(np.abs(2*run.model.grid.w@y[m:2*m]+y[-1]-2.55))}
        for key,value in expected.items():
            assert abs(dense[key]-value)<1e-12,(q,key,dense[key],value)
        assert dense['moisture_balance_max_sampled']<1e-7
        for bad in (0,-1,float('nan'),float('inf')):
            try:run.summary(sample_step_s=bad)
            except ValueError:pass
            else:raise AssertionError('Invalid step accepted')
        rows.append(dict(question=q,diagnostics=dense))
    if q==3:
        out=Path(tempfile.mkdtemp(prefix='smoke_output_',dir=Path(__file__).resolve().parent))
        export_run(run,out,chunk=137)
        audit=audit_book(out/'result2.xlsx',600)
        assert len(audit)==2 and all(a['data_rows']==600 for a in audit)
    print(json.dumps(dict(status='PASS',runs=rows),ensure_ascii=False,indent=2))

if __name__=='__main__':
    main()

```

B.8 完整附件读取

```
#!/usr/bin/env python3
""" 无隐式表头推断地读取 XLSX，并可断言数据行数。 """

from pathlib import Path

from openpyxl import load_workbook

def read_excel_rows(path, sheet=0, *, header=False, expected_rows=None):
    workbook = load_workbook(Path(path), read_only=True, data_only=True)
    worksheet = workbook.worksheets[sheet] if isinstance(sheet, int) else workbook[sheet]
    rows = [tuple(row) for row in worksheet.iter_rows(values_only=True)]
    while rows and not any(value is not None for value in rows[-1]):
        rows.pop()
    workbook.close()

    if header:
        if not rows:
            return []
        names = rows.pop(0)
        rows = [dict(zip(names, row)) for row in rows]
    if expected_rows is not None and len(rows) != expected_rows:
        raise ValueError(f" 工作表实际读取 {len(rows)} 行，期望 {expected_rows} 行")
    return rows
```

B.9 图表结构与布局检查

```
#!/usr/bin/env python3
""" 论文图的统一样式、字体选择与导出检查。 """

from __future__ import annotations

import logging
import unicodedata
import warnings
from pathlib import Path
from typing import Sequence

# -----
# 依赖资源定位
# -----

def _is_math_modeling_skill_root(path: Path) -> bool:
    return (
        (path / "SKILL.md").is_file()
        and (path / "VERSION").is_file()
        and (
            path
            / "references"
            / "roles"
            / " 编程手"
            / "scripts"
            / "plot_style.py"
        )
    )
```

```

        ).is_file()
    )

def _find_skill_root(path: Path) -> Path | None:
    directory = path.resolve().parent
    return next(
        (candidate for candidate in (directory, *directory.parents)
         if _is_math_modeling_skill_root(candidate)),
        None,
    )

SKILL_ROOT = _find_skill_root(Path(__file__))

# -----
# 色盲安全配色
# -----

PALETTE = {
    "primary": "#0072B2",
    "secondary": "#E69F00",
    "positive": "#009E73",
    "contrast": "#D55E00",
    "accent": "#CC79A7",
    "sky": "#56B4E9",
    "neutral": "#6B7280",
    "dark": "#222222",
}

COLOR_SEQUENCE = tuple(PALETTE[name] for name in (
    "primary",
    "secondary",
    "positive",
    "contrast",
    "accent",
    "sky",
    "neutral",
))

# -----
# 论文栏宽 (英寸)
# -----

WIDTHS_IN = {
    "single": 3.5,
    "double": 7.2,
    "report": 6.3,
}

# -----
# 字体选择
# -----

_CJK_SANS = (
    "Noto Sans CJK SC",

```

```

        "Source Han Sans SC",
        "Microsoft YaHei",
        "SimHei",
        "PingFang SC",
    )

def _available_fonts() -> set[str]:
    from matplotlib import font_manager
    return {item.name for item in font_manager.fontManager.ttflist}

def choose_font(language: str = "zh") -> str:
    """ 选择可用字体；中文字体缺失时给出警告并安全回退。"""
    if language not in {"zh", "en"}:
        raise ValueError("language 只能是 'zh' 或 'en'")
    available = _available_fonts()
    if language == "zh":
        for name in _CJK_SANS:
            if name in available:
                return name
        warnings.warn(
            " 未检测到常用中文字体，已回退到 DejaVu Sans；导出前必须检查中文和特殊符号是否缺字。",
            RuntimeWarning,
            stacklevel=2,
        )
    for name in ("Arial", "Helvetica", "DejaVu Sans"):
        if name in available:
            return name
    return "DejaVu Sans"

# -----
# 尺寸计算
# -----

def figure_size(width: str = "report", aspect: float = 0.62) -> tuple[float, float]:
    """ 按最终使用宽度返回英寸尺寸，避免在论文中二次大幅缩放。"""
    if width not in WIDTHS_IN:
        raise ValueError(f" 未知宽度方案: {width}")
    if aspect <= 0:
        raise ValueError("aspect 必须大于 0")
    width_in = WIDTHS_IN[width]
    return width_in, width_in * aspect

# -----
# 子图创建
# -----

def publication_subplots(
    nrows: int = 1,
    ncols: int = 1,
    *,
    width: str = "report",
    aspect: float = 0.62,

```

```

width_ratios: Sequence[float] | None = None,
height_ratios: Sequence[float] | None = None,
squeeze: bool = True,
):
    """ 按最终尺寸创建子图，并允许显式声明主次面板比例。"""
    import matplotlib.pyplot as plt

    if nrows < 1 or ncols < 1:
        raise ValueError("nrows 和 ncols 必须大于 0")
    if width_ratios is not None and len(width_ratios) != ncols:
        raise ValueError("width_ratios 数量必须与 ncols 一致")
    if height_ratios is not None and len(height_ratios) != nrows:
        raise ValueError("height_ratios 数量必须与 nrows 一致")
    gridspec_kw = {}
    if width_ratios is not None:
        gridspec_kw["width_ratios"] = list(width_ratios)
    if height_ratios is not None:
        gridspec_kw["height_ratios"] = list(height_ratios)
    return plt.subplots(
        nrows,
        ncols,
        figsize=figure_size(width, aspect),
        layout="constrained",
        gridspec_kw=gridspec_kw or None,
        squeeze=squeeze,
    )

# -----
# 面板标签
# -----

from collections.abc import Iterable as _Iterable

def add_panel_labels(
    axes: _Iterable,
    labels: Sequence[str] | None = None,
    *,
    x_offset_pt: float = -8.0,
    y_offset_pt: float = 1.0,
) -> None:
    """ 在各面板左上外侧添加小写粗体编号。"""
    axes_list = list(axes)
    panel_labels = list(labels) if labels is not None else [chr(97 + i) for i in range(len(axes_list))]
    if len(panel_labels) != len(axes_list):
        raise ValueError("labels 数量必须与 axes 数量一致")
    for axis, label in zip(axes_list, panel_labels):
        axis.annotate(
            label,
            xy=(0, 1),
            xycoords="axes fraction",
            xytext=(x_offset_pt, y_offset_pt),
            textcoords="offset points",
            ha="right",
            va="bottom",

```

```

        fontsize=8,
        fontweight="bold",
        annotation_clip=False,
    )

# -----
# 路径安全
# -----

def resolve_output_stem(output_stem: str | Path) -> Path:
    """ 解析导出路径，避免覆盖工具资源目录。 """
    stem = Path(output_stem).expanduser().resolve()
    if stem.suffix.lower() in {".svg", ".png", ".pdf"}:
        stem = stem.with_suffix("")
    if any(
        _is_math_modeling_skill_root(candidate)
        for candidate in (stem.parent, *stem.parent.parents)
    ):
        raise ValueError(" 图形产物必须写入 PROJECT_ROOT，不能写入 SKILL_ROOT")
    return stem

# -----
# 向后兼容：已被 tools/figure/ 新工具替代的函数
# 以下函数保留以兼容现有测试和外部代码。新代码请使用
# tools/figure/scripts/ 下的 setup_style、visual_qa、export_figure。
# -----

def apply_publication_style(language: str = "zh", width: str = "report") -> dict:
    """ 应用出版样式基线（向后兼容）。新代码请用 setup_style()。 """
    import matplotlib as mpl
    from cycler import cycler

    font_name = choose_font(language)
    size = figure_size(width)
    mpl.rcParams.update({
        "figure.figsize": size,
        "figure.constrained_layout.use": True,
        "figure.facecolor": "white",
        "axes.facecolor": "white",
        "font.family": "sans-serif",
        "font.sans-serif": [font_name, "Arial", "Helvetica", "DejaVu Sans"],
        "font.size": 7.5,
        "axes.titlesize": 8.0,
        "axes.titleweight": "normal",
        "axes.titlelocation": "left",
        "axes.titlepad": 4.0,
        "axes.labelsize": 8.0,
        "xtick.labelsize": 7.0,
        "ytick.labelsize": 7.0,
        "legend.fontsize": 7.0,
        "axes.linewidth": 0.7,
        "lines.linewidth": 1.1,
        "lines.markersize": 3.5,
        "patch.linewidth": 0.6,
    })

```

```

        "hatch.linewidth": 0.4,
        "xtick.major.size": 3.0,
        "ytick.major.size": 3.0,
        "xtick.major.width": 0.6,
        "ytick.major.width": 0.6,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.grid": False,
        "legend.frameon": False,
        "legend.handlelength": 1.6,
        "legend.borderaxespad": 0.4,
        "axes.unicode_minus": False,
        "axes.prop_cycle": cycler(color=COLOR_SEQUENCE),
        "svg.fonttype": "none",
        "pdf.fonttype": 42,
        "savefig.facecolor": "white",
        "savefig.dpi": 300,
    })
    return {"font": font_name, "size_inches": size, "colors": COLOR_SEQUENCE}

class _GlyphHandler(logging.Handler):
    def __init__(self) -> None:
        super().__init__()
        self.messages: list[str] = []

    def emit(self, record) -> None:
        message = record.getMessage()
        if "Glyph" in message and "missing from font" in message:
            self.messages.append(message)

def _labels_overlap(labels, renderer, axis: str) -> bool:
    boxes = [
        label.get_window_extent(renderer)
        for label in labels
        if label.get_visible() and label.get_text().strip()
    ]
    boxes.sort(key=(lambda box: box.x0) if axis == "x" else (lambda box: box.y0))
    if axis == "x":
        return any(left.x1 > right.x0 + 1 for left, right in zip(boxes, boxes[1:]))
    return any(lower.y1 > upper.y0 + 1 for lower, upper in zip(boxes, boxes[1:]))

def audit_layout(fig) -> list[str]:
    """ 在导出前检查缺字、画布外文字和相邻刻度重叠（向后兼容）。 """
    import matplotlib.text as mtext

    handler = _GlyphHandler()
    logger = logging.getLogger("matplotlib")
    logger.addHandler(handler)
    try:
        with warnings.catch_warnings(record=True) as caught:
            warnings.simplefilter("always")
            fig.canvas.draw()
    finally:

```

```

        logger.removeHandler(handler)

messages = handler.messages + [
    str(item.message)
    for item in caught
    if "Glyph" in str(item.message) and "missing from font" in str(item.message)
]
issues = [f" 缺字: {message}" for message in dict.fromkeys(messages)]
renderer = fig.canvas.get_renderer()
width, height = fig.bbox.width, fig.bbox.height
tick_ids = {
    id(label)
    for axis in fig.axes
    for label in (*axis.get_xticklabels(), *axis.get_yticklabels())
}
clipped = []
for text in fig.findobj(mtext.Text):
    if id(text) in tick_ids or not text.get_visible() or not text.get_text().strip():
        continue
    box = text.get_window_extent(renderer)
    if box.x0 < -1 or box.y0 < -1 or box.x1 > width + 1 or box.y1 > height + 1:
        clipped.append(text.get_text().replace("\n", " ")[:24])
if clipped:
    issues.append(" 文字可能超出画布: " + "、".join(dict.fromkeys(clipped)))
for index, axis in enumerate(fig.axes, start=1):
    if _labels_overlap(axis.get_xticklabels(), renderer, "x"):
        issues.append(f" 第 {index} 个坐标轴的 x 刻度标签重叠")
    if _labels_overlap(axis.get_yticklabels(), renderer, "y"):
        issues.append(f" 第 {index} 个坐标轴的 y 刻度标签重叠")
return issues

def _display_width(text: str) -> int:
    return sum(
        2 if unicodedata.east_asian_width(character) in {"W", "F"} else 1
        for character in text
    )

def _is_colorbar_axis(axis) -> bool:
    return axis.get_label() == "<colorbar>" or hasattr(axis, "_colorbar")

def audit_design(fig) -> list[str]:
    """ 检查可由对象结构确定的高风险设计 (向后兼容)。"""
    from matplotlib.container import BarContainer

    issues: list[str] = []
    data_axes = [axis for axis in fig.axes if not _is_colorbar_axis(axis)]
    colorbar_axes = [axis for axis in fig.axes if _is_colorbar_axis(axis)]
    colorbar_mappables = {
        id(axis._colorbar.mappable)
        for axis in colorbar_axes
        if hasattr(axis, "_colorbar") and getattr(axis._colorbar, "mappable", None) is not None
    }
    for index, axis in enumerate(data_axes, start=1):

```

```

for location in ("left", "center", "right"):
    title = axis.get_title(loc=location).strip()
    title_lines = title.splitlines()
    if title and (
        len(title_lines) > 1
        or max(_display_width(line) for line in title_lines) > 28
    ):
        issues.append(
            f" 第 {index} 个坐标轴标题过长；完整论述应移到图注，面板内只保留短标题"
        )
        break

legend = axis.get_legend()
if legend is not None and len(legend.get_texts()) > 5:
    issues.append(
        f" 第 {index} 个坐标轴图例超过 5 项；应直接标注、改为共享图例或拆分证据"
    )

for line in axis.lines:
    marker = line.get_marker()
    if marker in {None, "", " ", "None", "none"}:
        continue
    point_count = len(line.get_xdata(orig=False))
    if point_count > 25 and line.get_markevery() is None:
        issues.append(
            f" 第 {index} 个坐标轴对 {point_count} 个点逐点绘制标记；"
            " 应取消标记或设置 markevery"
        )
        break

for container in axis.containers:
    if not isinstance(container, BarContainer) or not container.patches:
        continue
    if container.orientation == "vertical":
        lower, upper = axis.get_ylim()
    else:
        lower, upper = axis.get_xlim()
    tolerance = max(abs(upper - lower), 1.0) * 1e-9
    if lower > tolerance or upper < -tolerance:
        issues.append(
            f" 第 {index} 个坐标轴的柱状图未从零开始；"
            " 若必须截断，应改用点图/区间图并显式说明"
        )
        break

for image in axis.images:
    values = image.get_array()
    if (
        getattr(values, "size", 0) <= 4
        and len(axis.texts) >= getattr(values, "size", 0)
        and id(image) in colorbar_mappables
    ):
        issues.append(
            f" 第 {index} 个坐标轴是已标数值的 2×2 小矩阵，"
            " 不应再使用冗余 colorbar"
        )

```

```

        break
    for legend in fig.legends:
        if len(legend.get_texts()) > 5:
            issues.append(" 整图共享图例超过 5 项；应直接标注、拆分证据或突出核心系列")
            break
    return issues

def _save_grayscale_preview(png_path: Path, dpi: int) -> Path:
    import matplotlib.image as image_io
    import numpy as np

    pixels = image_io.imread(png_path)
    rgb = pixels[..., :3]
    if pixels.shape[-1] == 4:
        alpha = pixels[..., 3:4]
        rgb = rgb * alpha + (1 - alpha)
    grayscale = np.dot(rgb, (0.2126, 0.7152, 0.0722))
    output = png_path.parent / "_qa" / f"{png_path.stem}_grayscale.png"
    output.parent.mkdir(parents=True, exist_ok=True)
    image_io.imsave(output, grayscale, cmap="gray", vmin=0, vmax=1, dpi=dpi)
    return output

def export_figure(
    fig,
    output_stem: str | Path,
    *,
    dpi: int = 300,
    grayscale_preview: bool = True,
    strict_layout: bool = True,
    strict_design: bool = True,
) -> dict[str, str]:
    """ 按固定物理尺寸导出（向后兼容）。新代码请用 tools/figure/scripts/export_figure.py。 """
    if dpi < 300:
        raise ValueError(" 论文图 PNG 的 dpi 不能低于 300")
    layout_issues = audit_layout(fig)
    if strict_layout and layout_issues:
        raise ValueError(" 版面预检未通过: " + "; ".join(layout_issues))
    design_issues = audit_design(fig)
    if strict_design and design_issues:
        raise ValueError(" 出版设计预检未通过: " + "; ".join(design_issues))
    stem = resolve_output_stem(output_stem)
    stem.parent.mkdir(parents=True, exist_ok=True)
    svg_path = stem.with_suffix(".svg")
    png_path = stem.with_suffix(".png")
    fig.savefig(svg_path)
    fig.savefig(png_path, dpi=dpi)
    outputs = {"svg": str(svg_path), "png": str(png_path)}
    if grayscale_preview:
        outputs["grayscale"] = str(_save_grayscale_preview(png_path, dpi))
    return outputs

__all__ = [
    "COLOR_SEQUENCE",

```

```

    "PALETTE",
    "WIDTHS_IN",
    "SKILL_ROOT",
    "add_panel_labels",
    "apply_publication_style",
    "audit_design",
    "audit_layout",
    "choose_font",
    "export_figure",
    "figure_size",
    "publication_subplots",
    "resolve_output_stem",
]

```

B.10 统一科研绘图风格

```

"""
Scientific figure style configuration.
Publication-grade matplotlib / seaborn style configuration.

```

应用出版级样式预设。支持 nature / ieee / science / general 四种期刊预设，支持中英文（lang='zh'/'en'），中文模式按优先级自动查找 Noto Sans CJK SC > Source Han Sans SC > SimHei > Microsoft YaHei 并修正负号渲染。SciencePlots 可选——装了就用，没装回退到内置等效预设。

Usage

```

from setup_style import setup_style

# Nature 单栏英文图
setup_style(journal='nature', lang='en')

# 中文期刊通用
setup_style(journal='general', lang='zh')

# 关闭 SciencePlots 强制用内置预设
setup_style(journal='ieee', use_sciplots=False)

```

CLI: ``python setup\_style.py --list-fonts`` 列出可用 CJK 字体。

```

"""
from __future__ import annotations

import argparse
import sys
import warnings

import matplotlib
import matplotlib.font_manager as fm
import matplotlib.pyplot as plt

```

```

# 期刊预设: figsize 单位为英寸, 对应该期刊单栏标称宽度
# Nature: 89mm = 3.5 in  双栏: 183mm = 7.2 in
# Science: 与 Nature 接近
# IEEE: 单栏 3.5 in, 双栏 7.16 in
JOURNAL_PRESETS = {

```

```

"nature": {
    "figure.figsize": (3.5, 2.625), # 4:3 ratio @ 89mm
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.family": "sans-serif",
    "font.sans-serif": ["Helvetica", "Arial", "DejaVu Sans"],
    "font.size": 7,
    "axes.labelsize": 8,
    "axes.titlesize": 8,
    "xtick.labelsize": 7,
    "ytick.labelsize": 7,
    "legend.fontsize": 7,
    "lines.linewidth": 1.0,
    "lines.markersize": 4,
    "axes.linewidth": 0.6,
    "xtick.major.width": 0.6,
    "ytick.major.width": 0.6,
    "xtick.minor.width": 0.4,
    "ytick.minor.width": 0.4,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "pdf.fonttype": 42,
    "ps.fonttype": 42,
    "svg.fonttype": "none",
},
"science": {
    "figure.figsize": (3.5, 2.625),
    "figure.dpi": 150,
    "savefig.dpi": 300,
    "font.family": "sans-serif",
    "font.sans-serif": ["Helvetica", "Arial", "DejaVu Sans"],
    "font.size": 7,
    "axes.labelsize": 7,
    "axes.titlesize": 8,
    "xtick.labelsize": 7,
    "ytick.labelsize": 7,
    "legend.fontsize": 6,
    "lines.linewidth": 1.0,
    "lines.markersize": 4,
    "axes.linewidth": 0.6,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "pdf.fonttype": 42,
    "ps.fonttype": 42,
    "svg.fonttype": "none",
},
"ieee": {
    "figure.figsize": (3.5, 2.5),
    "figure.dpi": 150,
    "savefig.dpi": 600,
    "font.family": "serif",
    "font.serif": ["Times New Roman", "Times", "DejaVu Serif"],
    "font.size": 8,
    "axes.labelsize": 8,
    "axes.titlesize": 9,
    "xtick.labelsize": 7,

```

```

        "ytick.labelsize": 7,
        "legend.fontsize": 7,
        "lines.linewidth": 1.0,
        "lines.markersize": 4,
        "axes.linewidth": 0.7,
        "axes.grid": False,
        "axes.spines.top": True,
        "axes.spines.right": True,
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
        "svg.fonttype": "none",
    },
    "general": {
        "figure.figsize": (5.0, 3.5),
        "figure.dpi": 150,
        "savefig.dpi": 300,
        "font.family": "sans-serif",
        "font.sans-serif": ["Arial", "Helvetica", "DejaVu Sans"],
        "font.size": 9,
        "axes.labelsize": 10,
        "axes.titlesize": 11,
        "xtick.labelsize": 9,
        "ytick.labelsize": 9,
        "legend.fontsize": 9,
        "lines.linewidth": 1.2,
        "lines.markersize": 5,
        "axes.linewidth": 0.8,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "pdf.fonttype": 42,
        "ps.fonttype": 42,
        "svg.fonttype": "none",
    },
}

```

中文字体优先级列表（按可用性 + 期刊接受度排序）

```

CJK_FONT_PRIORITY = [
    "Noto Sans CJK SC",
    "Noto Sans SC",
    "Source Han Sans SC",
    "Source Han Sans CN",
    "SimHei",
    "Microsoft YaHei",
    "PingFang SC",
    "Heiti SC",
    "WenQuanYi Zen Hei",
    "Arial Unicode MS",
]

```

```

CJK_SERIF_PRIORITY = [
    "Noto Serif CJK SC",
    "Noto Serif SC",
    "Source Han Serif SC",
    "Source Han Serif CN",
    "SimSun",
    "STSong",
]

```

```

    "Songti SC",
]

```

CJK_INSTALL_HINT = ""\

找不到任何中文字体。请安装 Noto CJK 字体之一：

```

Linux:    sudo apt install fonts-noto-cjk      # Debian/Ubuntu
          sudo dnf install google-noto-sans-cjk-fonts # Fedora/RHEL
macOS:    brew install --cask font-noto-sans-cjk-sc
          或下载: https://github.com/notofonts/noto-cjk/releases
Windows:  下载 https://github.com/notofonts/noto-cjk/releases
          解压后右键 .ttf/.otf 文件 -> " 为所有用户安装"

```

或者列出当前已安装的中文字体：

```

python setup_style.py --list-fonts
"""

```

```

def _available_fonts() -> set[str]:
    """ 返回 matplotlib 已索引的全部字体名集合。"""
    return {f.name for f in fm.fontManager.ttflist}

```

```

def list_cjk_fonts() -> list[str]:
    """ 返回系统上可用的 CJK 字体（按优先级排序）。"""
    available = _available_fonts()
    hits = []
    for f in CJK_FONT_PRIORITY + CJK_SERIF_PRIORITY:
        if f in available and f not in hits:
            hits.append(f)
    # 额外做一次包含中文关键词的扫描，捕获非标准命名的中文字体
    for f in available:
        lower = f.lower()
        if any(k in lower for k in ("cjk", "han", "songti", "yahei", "simhei", "simsun")):
            if f not in hits:
                hits.append(f)
    return hits

```

```

def configure_chinese_fonts(serif_for_zh: bool = False) -> str:
    """
    自动检测并配置中文字体；同时修正负号渲染。

    Args:
        serif_for_zh: True 时优先选用衬线中文字体（宋体类），用于中文期刊
            " 宋体正文 + Times New Roman 数字" 的混排约定。

    Returns:
        实际选用的中文字体名。

    Raises:
        RuntimeError: 系统未安装任何识别到的中文字体。
    """
    available = _available_fonts()
    priority = CJK_SERIF_PRIORITY + CJK_FONT_PRIORITY if serif_for_zh else CJK_FONT_PRIORITY

    chosen = None
    for f in priority:

```

```

        if f in available:
            chosen = f
            break

    if chosen is None:
        # 兜底：扫包含 cjk/han 等关键字的字体
        for f in available:
            lower = f.lower()
            if any(k in lower for k in ("cjk", "han", "song", "hei", "yahei", "kaiti")):
                chosen = f
                break

    if chosen is None:
        raise RuntimeError(CJK_INSTALL_HINT)

# 中文期刊常要求中文 + Times New Roman 混排：中文走中文字体，西文走 Times
plt.rcParams["font.family"] = ["sans-serif"] if not serif_for_zh else ["serif"]
if serif_for_zh:
    plt.rcParams["font.serif"] = [chosen, "Times New Roman", "Times", "DejaVu Serif"]
else:
    plt.rcParams["font.sans-serif"] = [chosen, "Arial", "Helvetica", "DejaVu Sans"]
# 修正负号 unicode minus 在某些中文字体里渲染成方框的问题
plt.rcParams["axes.unicode_minus"] = False
return chosen

def _try_sciencplots(journal: str) -> bool:
    """If SciencePlots is installed, apply its style stack; otherwise return False."""
    try:
        import scienceplots # noqa: F401
    except ImportError:
        return False

    # SciencePlots 风格栈：基础 + 期刊变体
    stack = ["science"]
    if journal == "nature":
        stack.append("nature")
    elif journal == "ieee":
        stack.append("ieee")
    # 关掉 LaTeX 渲染避免环境缺 LaTeX 时崩溃；中文模式必须关
    stack.append("no-latex")
    try:
        plt.style.use(stack)
        return True
    except OSError as e:
        warnings.warn(f"SciencePlots style stack failed: {e}; fallback to builtin.")
        return False

def setup_style(
    journal: str = "general",
    lang: str = "en",
    use_sciplots: bool = True,
    serif_for_zh: bool = False,
    constrained_layout: bool = True,
) -> dict:

```

```

"""
应用出版级样式预设。

Args:
    journal: 'nature' | 'science' | 'ieee' | 'general'
    lang: 'en' | 'zh' —中文模式自动配置中文字体并修正负号
    use_sciplots: 优先尝试 SciencePlots; 不可用则回退到内置预设
    serif_for_zh: 中文模式下使用宋体类衬线字体 (中文期刊常约定)
    constrained_layout: 默认 True——全局开启 constrained_layout 自适应排版,
        从源头减少标题/轴标签被裁、图例压数据、子图互相重叠。需要手动
        subplots_adjust 或某些 colorbar 写法时可传 False 关闭。
Returns:
    dict 包含 keys: journal / lang / sciplots_used / cjk_font / constrained_layout
"""
if journal not in JOURNAL_PRESETS:
    raise ValueError(f"Unknown journal preset: {journal}. "
                     f"Choose from {sorted(JOURNAL_PRESETS)}")

sciplots_used = False
if use_sciplots:
    sciplots_used = _try_sciencplots(journal)

# 内置预设始终在 SciencePlots 之上覆盖一遍, 确保关键参数 (fonttype、字号) 落实
plt.rcParams.update(JOURNAL_PRESETS[journal])

# 默认开启自适应排版: 从源头减少文字遮盖 / 裁切 / 子图重叠
plt.rcParams["figure.constrained_layout.use"] = constrained_layout

# 全模式默认修正负号: 避免所选字体缺 U+2212 时负号渲染成方框 (一种乱码)。
# 用 ASCII hyphen-minus 代替真减号, 几乎所有字体都含, 最稳妥。
plt.rcParams["axes.unicode_minus"] = False

cjk_font = None
if lang == "zh":
    cjk_font = configure_chinese_fonts(serif_for_zh=serif_for_zh)
elif lang != "en":
    raise ValueError(f"lang must be 'en' or 'zh', got {lang!r}")

return {
    "journal": journal,
    "lang": lang,
    "sciplots_used": sciplots_used,
    "cjk_font": cjk_font,
    "constrained_layout": constrained_layout,
}

def _cli() -> int:
    p = argparse.ArgumentParser(description="scipilot-figure-skill style setup")
    p.add_argument("--list-fonts", action="store_true",
                   help="列出当前系统上可用的 CJK 字体")
    p.add_argument("--test", action="store_true",
                   help="应用预设并打印 rcParams 关键值")
    p.add_argument("--journal", default="general",
                   choices=list(JOURNAL_PRESETS))
    p.add_argument("--lang", default="en", choices=["en", "zh"])

```

```

p.add_argument("--no-sciplots", action="store_true")
p.add_argument("--serif-zh", action="store_true")
args = p.parse_args()

if args.list_fonts:
    fonts = list_cjk_fonts()
    if fonts:
        print(" 已检测到的 CJK 字体 (按优先级排序): ")
        for f in fonts:
            print(f"    - {f}")
    else:
        print(" 未检测到任何 CJK 字体。")
        print(CJK_INSTALL_HINT)
    return 0

if args.test:
    info = setup_style(journal=args.journal, lang=args.lang,
                      use_sciplots=not args.no_sciplots,
                      serif_for_zh=args.serif_zh)
    print(f"applied: {info}")
    for k in ("figure.figsize", "font.family", "font.size",
             "axes.labelsize", "pdf.fonttype", "axes.unicode_minus"):
        print(f"    {k} = {plt.rcParams[k]!r}")
    return 0

p.print_help()
return 0

if __name__ == "__main__":
    sys.exit(_cli())

```

B.11 多格式图表导出

```

"""
Scientific figure export utilities.
Unified figure export to multiple formats at exact final size.

- Vector preferred: PDF / SVG / EPS for line/bar/scatter (lossless, journal-friendly).
- Raster for photos / micrographs: PNG / TIFF at >= 300 DPI; never JPEG for data figures.
- Embeds TrueType fonts (fonttype 42) so journals don't reject Type-3 PDFs.
- Optional grayscale preview to sanity-check colorblind safety.

Usage
-----

from export_figure import export_figure
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot([0,1,2],[3,1,4])

paths = export_figure(
    fig,
    basename="figs/fig1_main",
    formats=["pdf", "svg", "png"],
    size_inches=(3.5, 2.625),    # 强制成 Nature 单栏尺寸

```

```

        dpi=600,
        grayscale_preview=True,
    )
    # -> ['figs/fig1_main.pdf', 'figs/fig1_main.svg',
    #     'figs/fig1_main.png', 'figs/fig1_main_grayscale.png']

CLI: ``python export_figure.py demo`` 生成一张演示图并多格式导出。
"""

from __future__ import annotations

import argparse
import os
import sys
from typing import Iterable

import matplotlib.pyplot as plt

VECTOR_FORMATS = {"pdf", "svg", "eps"}
RASTER_FORMATS = {"png", "tiff", "tif", "jpg", "jpeg"}
SUPPORTED_FORMATS = VECTOR_FORMATS | RASTER_FORMATS

def _ensure_parent(path: str) -> None:
    parent = os.path.dirname(os.path.abspath(path))
    if parent and not os.path.exists(parent):
        os.makedirs(parent, exist_ok=True)

def export_figure(
    fig,
    basename: str,
    formats: Iterable[str] | None = None,
    dpi: int = 300,
    size_inches: tuple[float, float] | None = None,
    grayscale_preview: bool = False,
    tight: bool = True,
    pad_inches: float = 0.05,
    transparent: bool = False,
) -> list[str]:
    """
    Export a matplotlib Figure to one or more formats at exact final size.

    Args:
        fig: matplotlib Figure object.
        basename: 输出路径前缀（不含扩展名）；可包含子目录，自动创建。
        formats: list/tuple of extensions, e.g. ['pdf', 'svg', 'png'].
            Default: ['pdf', 'svg', 'png'].
        dpi: raster 格式分辨率；建议 300（普通）/ 600（IEEE 等）。
        size_inches: (width, height) 英寸；指定后会 fig.set_size_inches() 强制
            最终尺寸。强烈建议传入——保证导出后不必在 Word/LaTeX 里二次缩放。
        grayscale_preview: 额外生成一张 _grayscale.png 供色盲安全检查。
        tight: 是否走 bbox_inches='tight'（裁掉留白）。
        pad_inches: tight 模式下保留的边距（英寸）。
        transparent: 透明背景（PPT/海报可能需要）。

```

```

Returns:
    实际写出的文件路径列表。
"""
if formats is None:
    formats = ("pdf", "svg", "png")
formats = [f.lower().lstrip(".") for f in formats]
unknown = [f for f in formats if f not in SUPPORTED_FORMATS]
if unknown:
    raise ValueError(f"Unsupported formats: {unknown}. "
                     f"Supported: {sorted(SUPPORTED_FORMATS)}")

if size_inches is not None:
    if len(size_inches) != 2:
        raise ValueError("size_inches must be (width, height)")
    fig.set_size_inches(*size_inches)

# 强制嵌入 TrueType 字体 (fonttype 42); 多家期刊明确拒绝 Type-3 PDF。
plt.rcParams["pdf.fonttype"] = 42
plt.rcParams["ps.fonttype"] = 42
plt.rcParams["svg.fonttype"] = "none" # 文本仍可编辑, 期刊通常更欢迎

saved: list[str] = []
for fmt in formats:
    if fmt in {"jpg", "jpeg"}:
        print(f"[scipilot-figure-skill] WARNING: skipping {fmt} — "
              "JPEG is lossy and unsuitable for line/text figures.",
              file=sys.stderr)
        continue
    path = f"{basename}.{fmt}"
    _ensure_parent(path)
    kwargs: dict = {
        "bbox_inches": "tight" if tight else None,
        "pad_inches": pad_inches,
        "transparent": transparent,
    }
    if fmt in RASTER_FORMATS:
        kwargs["dpi"] = dpi
    fig.savefig(path, **kwargs)
    saved.append(path)
    print(f"[scipilot-figure-skill] wrote {path}")

if grayscale_preview:
    gray_path = _grayscale_from(fig, basename, dpi=dpi)
    if gray_path:
        saved.append(gray_path)
return saved

def _grayscale_from(fig, basename: str, dpi: int) -> str | None:
    """
    导出灰度预览版用于色盲安全检查。
    优先用 PIL 转灰度; 找不到 PIL 时退化为重新画图 (关闭颜色)。
    """
    try:
        from PIL import Image
    except ImportError:

```

```

        print("[scipilot-figure-skill] Pillow not available; "
              "grayscale preview skipped.", file=sys.stderr)
        return None

    png_path = f"{basename}.png"
    _ensure_parent(png_path)
    fig.savefig(png_path, dpi=dpi, bbox_inches="tight")

    gray_path = f"{basename}_grayscale.png"
    Image.open(png_path).convert("L").save(gray_path)
    print(f"[scipilot-figure-skill] wrote {gray_path} (grayscale preview)")
    return gray_path

def _demo(out_basename: str) -> None:
    """Generate a small demo figure for quick smoke testing."""
    import numpy as np
    rng = np.random.default_rng(7)
    x = np.linspace(0, 10, 50)
    y1 = np.sin(x) + rng.normal(0, 0.1, x.size)
    y2 = np.cos(x) + rng.normal(0, 0.1, x.size)

    fig, ax = plt.subplots(figsize=(3.5, 2.625))
    ax.plot(x, y1, label="sin", marker="o", markersize=3)
    ax.plot(x, y2, label="cos", marker="s", markersize=3, linestyle="--")
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.legend(frameon=False)

    paths = export_figure(
        fig, out_basename,
        formats=["pdf", "svg", "png"],
        size_inches=(3.5, 2.625),
        dpi=300,
        grayscale_preview=True,
    )
    print("\nDemo done. Files:")
    for p in paths:
        print(f"  {p}")

def _cli() -> int:
    p = argparse.ArgumentParser(description="scipilot-figure-skill figure exporter")
    p.add_argument("cmd", choices=["demo"], help="demo: 跑一张演示图导出 4 种格式")
    p.add_argument("--out", default="./scipilot_demo",
                  help="输出 basename (默认 ./scipilot_demo)")
    args = p.parse_args()
    if args.cmd == "demo":
        _demo(args.out)
    return 0

if __name__ == "__main__":
    sys.exit(_cli())

```

B.12 输入与运行环境记录

```
#!/usr/bin/env python3
""" 记录计算环境与输入输出，避免覆盖工具资源。 """

import argparse
import hashlib
import importlib.metadata
import json
import platform
import sys
from datetime import datetime, timezone
from pathlib import Path

SKILL_ROOT = Path(__file__).resolve().parents[2] / '.agents/skills/math-modeling'

def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1024 * 1024), b''):
            digest.update(chunk)
    return digest.hexdigest()

def build_manifest(
    inputs,
    seed,
    parameters,
    command,
    packages,
    runtime_name="python",
    runtime_version=None,
    dependencies=None,
):
    files = []
    for item in inputs:
        path = Path(item).resolve()
        if not path.is_file():
            raise FileNotFoundError(f" 输入文件不存在: {path}")
        files.append({"path": str(path), "sha256": sha256_file(path), "bytes": path.stat().st_size})

    versions = dict(dependencies or {})
    if runtime_name == "python":
        for package in packages:
            try:
                versions[package] = importlib.metadata.version(package)
            except importlib.metadata.PackageNotFoundError:
                versions[package] = None
        runtime_version = runtime_version or sys.version.split()[0]

    return {
        "schema_version": 1,
        "created_at_utc": datetime.now(timezone.utc).isoformat(),
        "random_seed": seed,
        "input_files": files,
    }
```

```

        "runtime": {
            "name": runtime_name,
            "version": runtime_version,
            "platform": platform.platform(),
            "dependencies": versions,
        },
        "key_parameters": parameters,
        "reproduce_command": command,
    }

def _is_within(path: Path, parent: Path) -> bool:
    try:
        path.relative_to(parent)
        return True
    except ValueError:
        return False

def resolve_output(project_root, relative_output):
    project = Path(project_root).resolve()
    if _is_within(project, SKILL_ROOT):
        raise ValueError("PROJECT_ROOT 不能位于 SKILL_ROOT 内部")
    output = (project / relative_output).resolve()
    if not _is_within(output, project):
        raise ValueError("输出路径必须位于 PROJECT_ROOT 内部")
    if _is_within(output, SKILL_ROOT):
        raise ValueError("输出路径不能位于 SKILL_ROOT 内部")
    return output

def main() -> int:
    parser = argparse.ArgumentParser(description="生成数学建模复现清单")
    parser.add_argument("--project-root", required=True)
    parser.add_argument("--input", action="append", default=[])
    parser.add_argument("--seed", required=True, type=int)
    parser.add_argument("--parameters", default="{}", help="JSON 对象")
    parser.add_argument("--command", required=True)
    parser.add_argument("--package", action="append", default=[])
    parser.add_argument("--runtime", choices=["python", "matlab"], default="python")
    parser.add_argument("--runtime-version")
    parser.add_argument("--dependencies", default="{}", help="依赖或 MATLAB 工具箱版本 JSON 对象")
    parser.add_argument("--output", default="results/复现清单.json")
    parser.add_argument("--overwrite", action="store_true")
    args = parser.parse_args()

    output = resolve_output(args.project_root, args.output)
    if output.exists() and not args.overwrite:
        print(f"错误: 输出已存在, 未覆盖: {output}", file=sys.stderr)
        return 1
    parameters = json.loads(args.parameters)
    if not isinstance(parameters, dict):
        raise ValueError("--parameters 必须是 JSON 对象")
    dependencies = json.loads(args.dependencies)
    if not isinstance(dependencies, dict):
        raise ValueError("--dependencies 必须是 JSON 对象")

```

```

manifest = build_manifest(
    args.input,
    args.seed,
    parameters,
    args.command,
    args.package,
    runtime_name=args.runtime,
    runtime_version=args.runtime_version,
    dependencies=dependencies,
)
output.parent.mkdir(parents=True, exist_ok=True)
output.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
print(output)
return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

B.13 Python 依赖版本

```

numpy==2.2.6
scipy==1.15.3
pandas==2.3.3
openpyxl==3.1.5
matplotlib==3.10.9
Pillow==12.3.0
fonttools==4.64.0
pymupdf==1.28.2

```

C 大型结果表与补充材料

C.1 全过程数据的范围与读取

表17列出四个工作簿的输出范围，数据行数不含表头。大型结果保存在电子支撑材料 `results/`中，避免在纸面重复逐秒数据。题目要求的六张摘录表均保留在四问正文，原始数值表同时保存为 `table1.csv` 至 `table6.csv`。

表 17: 完整结果文件的时间范围与径向输出

文件	时间范围/s	每表数据行	径向位置/cm
<code>result1.xlsx</code>	1–1800	1800	0、0.1、...、2.0；温度与水分两表
<code>result2.xlsx</code>	1–206902	206902	0、0.1、...、2.0；温度与水分两表
<code>result3.xlsx</code>	60–206902	3449	0、0.1、...、2.0；含水率
<code>result4.xlsx</code>	60–183927	3066	0、0.1、...、1.9 及实际表面；含水率

问题一、二按 1 秒间隔输出；问题二覆盖整个烘干过程，正文题表仍只列 0.5 至 3.0 小时。问题三、四按 60 秒间隔输出，并分别补入严格整秒终点 206902 秒和 183927 秒。四位小数仅用于展示，终点判断采用未舍入数值。

固定域按物理半径采样。问题四的实际表面列对应 $R(t)$ ，当固定位置 $r > R(t)$ 时留空，题表用破折号表示；工作簿 4 共有 20099 个此类域外位置。固定位置与材料轨迹 $r = xR(t)$ 有不同含义，应按所需对象读取。

C.2 补充轨迹与验证数据

`results/`中的过程数组与`q1_history.csv`、`q3_history.csv`、`q4_history.csv` 保留未舍入场及过程指标。网格、时间和敏感性比较分别见`grid_comparison.csv`、`time_comparison.csv`、`sensitivity.csv`，移动域基准详见`moving_domain_benchmark.json` 与`moving_domain_convergence.csv`。

`figures/`保留 14 组原始输入、过程及结果图的 PDF、SVG 和 PNG 文件；正文选用其中 12 幅回答四问与数值检验。完整源程序已列于附录 B，运行条件与入口见附录A。